

Rendus soutenance finale:

25/05/2026

WHAT'S NEXT?

**Hadrien Beaurepaire
Ghali Benani
Nael Belhadj Amara
Julia Guimaraes
Melissa Gay**

Plan soutenance:

- 1) Présentation bref du projet et du groupe: 1 min

Ghali

2) Récapitulatif des attentes du projet: (support: cdet) 1 minutes

Julia

3) Installation de l'exécutable + manuel d'installation: 1 min

Julia

4) Démonstration du projet: 14 min:

5) Présentation du menu: 3 min

Nael

6) Présentation des salles 11 min

Melissa + Hadrien + Nael + Ghali

7) Manuel de désinstallation + explications des failles 1 min

Julia

8) Présentation du site web: 1 min

Melissa

9) Conclusion: 1 min

Ghali

Introduction:

Dans le cadre de notre première année de prépa à EPITA nous avons été chargés d'exécuter le projet J3D. En suivant les consignes nous avons créé "What's Next".

Le projet J3D:

Pour mieux comprendre la création de notre jeu, il est important de comprendre le contexte qui nous est imposé.

Ce projet s'inscrit dans le cadre de la SAE J3D de première année à l'EPITA, dont la consigne est de concevoir et de développer, en groupe de quatre à cinq étudiants, un jeu vidéo tout au long de l'année universitaire. Ce projet informatique constitue un travail de longue durée visant à mettre en application les connaissances acquises en cours et en travaux pratiques de programmation, tout en mobilisant des compétences complémentaires nécessaires à la réalisation d'un projet concret.

Conformément à la consigne, le jeu doit être développé en Python, utiliser le framework graphique PyGame, intégrer une forme d'intelligence artificielle simple ainsi qu'un mode multijoueur en réseau. Le projet doit également être fonctionnel sur Windows 10/11 et respecter les contraintes techniques et organisationnelles définies par l'encadrement pédagogique. Cela inclut la préparation de deux soutenances en cours d'année et une soutenance finale avec des rendues respectives à chacune d'elles.

Pour nous, la problématique centrale du projet est la suivante :

Comment concevoir et développer un jeu vidéo original répondant aux contraintes pédagogiques, techniques et temporelles imposées par la SAE J3D, tout en proposant une expérience de jeu cohérente, immersive et engageante pour les joueurs ?

Cette problématique implique de concilier les exigences académiques (langage, framework, fonctionnalités obligatoires) avec des choix de game design pertinents, une organisation efficace du travail en équipe et une architecture logicielle adaptée, notamment pour la gestion du réseau et de l'intelligence artificielle.

Ce contexte vise un résultat de novices, dans le domaine des jeux vidéos, se focalisant alors sur le parcours et les compétences que l'on acquiert en chemin.

L'identité de "What's Next?":

L'identité de notre jeu est le fruit d'un travail collectif. Dès les premières réunions de groupe, nous avons fait le choix de construire notre projet autour des préférences et des sensibilités de chaque membre, afin que "What's Next?" nous ressemble véritablement. C'est pourquoi il nous semble indispensable de présenter notre équipe avant d'aborder les aspects techniques du projet : comprendre qui nous sommes en tant que joueurs permet de mieux comprendre les choix que nous avons faits en tant que développeurs.

Présentation des membres de notre équipe:

Notre équipe, baptisée Boneca Ambalabu, est composée de cinq membres aux profils et aux sensibilités ludiques très différents. Afin de mieux cerner notre identité collective en tant que joueurs, nous avons recensé dès le début du projet deux informations clés pour chaque membre : **le type de jeu préféré (TJP) et le jeu préféré (JP).**

Ces informations ont été recueillies lors de nos premières réunions et ont constitué le point de départ de nos réflexions sur le type de jeu que nous souhaitons créer.

Ghali est un joueur orienté action intense, dont le jeu préféré appartient au genre survival horror. Cette sensibilité pour les univers sombres, stressants et à l'atmosphère soignée a influencé notre réflexion sur la tension narrative du jeu.

Hadrien est un joueur RPG, fan d'Overwatch. Son goût pour les univers riches en lore, les mécaniques de personnages distinctes et le jeu en équipe a orienté nos choix en matière de game design coopératif.

Julia est une joueuse de simulation de vie, dont le jeu de référence est Les Sims 4. Sa sensibilité pour les interactions entre personnages, la narration quotidienne et l'attachement émotionnel aux environnements a contribué à l'atmosphère intimiste de certaines salles du jeu.

Melissa est également une joueuse RPG, dont le jeu préféré est Detroit: Become Human. Son goût pour les récits à choix moraux, les fins multiples et les personnages profonds a directement influencé la conception de la salle finale et la dimension narrative de "What's Next?".

Nael est un joueur FPS, dont le jeu de référence est Days Gone. Son appétence pour l'action, le gameplay dynamique et les environnements post-apocalyptiques a nourri notre réflexion sur les mécaniques de tension et de survie présentes dans les donjons.

La diversité de nos profils de joueurs constitue à la fois une richesse et un défi : elle nous a permis d'envisager "What's Next?" sous des angles très différents, tout en nécessitant de nombreuses discussions et compromis pour aboutir à une vision commune cohérente.



Ghali Hadrien Julia

TJP: ultra kill RPG simulation de vie **JP:** survival horror overwatch the sims 4



Melissa Nael

TJP: RPG FPS

JP: Detroit become human Days Gone

Choix du nom de l'équipe:

Le choix du nom de notre équipe s'est fait de manière spontanée, reflétant notre volonté de ne pas surcharger cette décision d'une réflexion trop formelle.

Ayant consacré l'essentiel de nos premières réunions à définir l'identité du jeu lui-même, nous avons délibérément gardé ce choix simple et rapide. C'est ainsi que, lors d'un moment de détente entre les discussions techniques, le nom "**Boneca Ambalabu**" a été proposé par Nael.

Ce nom a immédiatement fait l'unanimité : il reflète l'esprit détendu que nous souhaitions insuffler à notre collaboration, et constitue un contrepoint volontairement léger au sérieux du projet que nous menions.

Choix du type de jeu:

Le choix du type de jeu a constitué la première grande décision collective du projet, et probablement la plus structurante. Plutôt que de partir d'une idée préconçue, nous avons fait le choix méthodique de croiser nos profils de joueurs afin d'identifier les points de convergence entre nos sensibilités respectives.

Plusieurs constats ont rapidement émergé de ces discussions.

Premièrement, la majorité des membres du groupe apprécie les **jeux à forte dimension narrative**, dans lesquels les choix des joueurs influencent le déroulement de l'histoire.

Deuxièmement, aucun d'entre nous ne possédait d'expérience préalable significative en développement de jeux vidéo, ni en création graphique avancée. Il nous semblait donc plus judicieux de viser un projet techniquement maîtrisable plutôt que de nous lancer dans des ambitions démesurées qui auraient compromis la qualité du rendu final.

C'est à partir de ces constats que nous avons convergé vers un **jeu de puzzle coopératif en deux dimensions, jouable en réseau local**.

La **dimension 2D** s'est imposée comme le choix le plus adapté à notre niveau technique : elle permettait de concentrer nos efforts sur la qualité du gameplay et de la narration plutôt que sur la complexité graphique.

Le **format puzzle**, quant à lui, correspondait à notre souhait commun de créer un jeu dans lequel la réflexion prime sur les réflexes, et où chaque salle constitue un défi intellectuel à résoudre ensemble.

La **dimension coopérative** a été au cœur de notre vision dès le départ. Nous souhaitions créer une expérience de jeu dont la qualité dépend directement de la communication et de la coordination entre les deux joueurs.

Cette approche offre une rejouabilité naturelle : l'expérience vécue varie sensiblement selon le coéquipier avec lequel on joue, les décisions prises collectivement et les stratégies adoptées face aux énigmes. C'est cette **multiplicité des expériences** possibles qui nous a convaincus de la pertinence de ce format.

Enfin, l'intégration d'une intelligence artificielle s'est imposée comme une conséquence logique de ces choix : dans un jeu coopératif où les deux joueurs doivent s'entraider pour

progresser, la présence d'**ennemis contrôlés par une IA** renforce la tension, encourage la coordination et justifie narrativement les obstacles rencontrés dans les donjons.

Choix du nom du jeu:

C'est une fois le type de jeu défini que nous nous sommes penchés sur le nom du jeu lui-même. Ce choix était important car il devait incarner à la fois l'identité visuelle, l'atmosphère et la philosophie de l'expérience que nous voulions proposer.

Notre jeu repose sur une **progression successive** de salles, chacune constituant une nouvelle énigme à résoudre avant de pouvoir accéder à la suivante.

Cette structure génère naturellement une forme de suspense et de curiosité : **qu'est-ce qui attend les joueurs de l'autre côté de la prochaine porte ? Quel nouveau défi, quelle nouvelle surprise, quelle nouvelle révélation narrative les attend ?**

C'est cette sensation précise — cette question que l'on se pose inévitablement à la fin de chaque salle — qui a inspiré le nom "**What's Next?**".

Simple, direct et universel, ce titre en anglais capte immédiatement l'essence du jeu : une expérience fondée sur la curiosité, l'anticipation et le désir d'aller toujours plus loin. Il s'adresse à tous les joueurs curieux, ceux qui ne peuvent pas s'empêcher de vouloir découvrir ce qui les attend au-delà de chaque obstacle.

Ce nom résonne également à un niveau plus profond : dans un jeu où les choix des joueurs influencent l'issue de certaines salles et déterminent la fin de l'aventure, la question "What's Next?" n'est pas seulement posée par le jeu aux joueurs — elle est aussi posée par les joueurs à eux-mêmes, à chaque décision qu'ils prennent ensemble.

Utilisateurs cibles:

L'identité de "**What's Next?**" est aussi réfléchie pour un certain public. Cette distinction nous a permis de faire des choix plus orientés tout au long de notre projet.

Les utilisateurs cibles de **WHAT'S NEXT?** sont principalement des joueurs et joueuses appréciant les expériences coopératives riches en réflexion et en narration.

Le jeu s'adresse à un public adolescent et jeune adulte, habitué aux jeux vidéo mais ne cherchant pas nécessairement une grande exigence technique ou des réflexes précis. Il vise avant tout des joueurs qui aiment communiquer, collaborer et prendre des décisions ensemble, que ce soit entre amis, en couple ou entre membres d'une même famille.

Sensibles aux univers immersifs, aux histoires à dimension morale et aux mécaniques de gameplay asymétriques, ces utilisateurs trouveront dans **WHAT'S NEXT?** une expérience originale qui valorise l'entraide, la coordination et les choix partagés, tout en proposant une réflexion plus profonde sur la dépendance, la confiance et les conséquences de nos actions.

Lore:

Notre identité après le type d'utilisateurs est ancrée dans le lore.

Pour une exécution du projet plus facile il nous a été impératif de créer un lore impactant.

Notre jeu suit une histoire, un lore précis qui guide nos choix. Voici l'histoire que nous avons imaginé après plusieurs temps de réflexions pour orienter le jeu:

Dans un paisible village, deux amis passent leurs journées à aider les habitants et à explorer les environs. Rien ne laissait présager que cette journée serait différente des autres.

Alors qu'ils traversent la place du marché, une vieille dame les interpelle. Elle semble inquiète : un objet précieux, une ancienne relique, a disparu non loin du village. Elle leur promet une récompense s'ils acceptent d'aller la récupérer.

Les deux amis partent donc sur le chemin forestier qui mène aux ruines voisines. Après quelques épreuves, ils parviennent à mettre la main sur l'objet. Mais sur le chemin du retour, alors qu'ils traversent un passage étroit entre des rochers, une embuscade se déclenche.

Le sol s'effondre sous leurs pieds.

Ils chutent dans l'obscurité... et se réveillent dans un immense complexe souterrain composé de donjons reliés entre eux.

En explorant ces lieux, ils découvrent un étrange cristal vert, incrusté dans la roche et pulsant d'une lueur surnaturelle. Poussés par la curiosité — et par la nécessité de survivre — ils en consomment un fragment.

Très vite, ils sentent leur corps changer.

Force, vitesse, pouvoirs étranges... le cristal leur offre des capacités qu'aucun humain ne devrait posséder. Grâce à lui, ils peuvent résoudre les mécanismes des donjons et affronter les dangers qui les guettent.

Mais ce pouvoir a un prix.

Plus ils avancent, plus ils ressentent le besoin d'en consommer à nouveau. Une sensation de manque s'installe. Le cristal n'est plus seulement un outil : il devient une obsession.

Pour retrouver la surface — et comprendre le lien entre la relique, la vieille dame, et ce monde souterrain — ils devront continuer à plonger toujours plus profondément dans les donjons.

Ensemble.

En ayant cette histoire en tête, nous avons choisi de customiser notre jeu avec des graphismes, musiques, site web et FX customs, afin de donner un rendu personnalisé à notre projet.

Graphismes:

Le graphisme du jeu est passé par plusieurs étapes différentes.

Il nous a été difficile de conceptualiser une vision graphique de notre jeu car aucun de nous n'est expert en dessin. Donc nous avons fait du mieux qu'on pouvait pour quand

même rendre quelque chose faite par nous et personnalisé. C'est pour cela qu'au début nous nous sommes beaucoup aidés de l'IA pour nous aider à visualiser nos idées.

Les premiers sprites et illustrations ont été faits en expliquant à l'intelligence artificielle l'identité visuelle que nous voulions avoir.

C'est pour cela aussi que nous avons beaucoup changé de personnages au cours de notre projet. On a fait beaucoup de tests et nous avons compris au fur et à mesure ce que nous voulions et surtout comment nous devions le faire.

Réalisations de Hadrien:

Personnages:

Pour ce qui est des personnages, nous avons au début surtout développé les personnages principaux qui représentent les deux joueurs. Afin de les créer nous avons utilisé l'IA ainsi qu'une tablette graphique. L'IA a été pratique afin de nous faire un premier modèle qui corresponde parfaitement à nos attentes et que l'on pouvait modifier rapidement pour l'adapter à un modèle qui plaisait à tout le monde.

Une fois un premier modèle créé pour chaque personnage nous avons utilisé une tablette graphique et continué à utiliser l'IA afin de faire les différents sprites.

Le défi ici était d'avoir un maximum de sprites différents afin que notre jeu soit fluide. Par exemple, lors d'un changement de direction de droite à gauche, il faut que le sprite change pour montrer le changement de direction plus fluidement, pareil pour aller de haut en bas ou interagir.

Deuxième partie de la création des sprites:

La création des sprites s'est structurée en quatre étapes distinctes, chacune répondant à des contraintes à la fois esthétiques et techniques liées au développement du jeu. La première étape a consisté à concevoir un modèle de base pour chacun des deux personnages jouables.

L'objectif était de produire des visuels cohérents avec l'univers du jeu tout en garantissant une différenciation claire entre les entités.

Nous avons donc défini une direction artistique orientée vers un style médiéval-fantastique, en adéquation avec l'environnement de donjon et la narration globale du jeu. Afin d'assurer une lisibilité immédiate en jeu, les personnages ont été différenciés par une palette de couleurs distincte (vert pour l'un, bleu pour l'autre), tout en conservant des éléments visuels communs (cape, épée, proportions similaires) pour maintenir une cohérence stylistique.

Cette phase a abouti à la création des premiers modèles statiques servant de base à toutes les animations ultérieures.

La deuxième étape a porté sur la définition des états d'animation et des directions de déplacement. Dans une optique de fluidité et de réactivité, il a été nécessaire d'anticiper l'ensemble des orientations possibles du personnage. Nous avons ainsi implémenté un système de déplacement en 8 directions : haut, bas, gauche, droite, ainsi que les quatre diagonales correspondantes.

Chaque direction nécessite une animation spécifique afin de conserver une cohérence

visuelle du mouvement. En parallèle, nous avons également défini les animations d'action, notamment l'attaque à l'épée.

Pour des raisons de simplicité d'implémentation et de cohérence gameplay, l'animation d'attaque a également été réutilisée pour certaines interactions avec l'environnement, évitant ainsi la création de nouvelles séquences dédiées.

La troisième étape a consisté en la production et le traitement des frames d'animation. Afin d'obtenir un rendu fluide, nous avons fixé à cinq le nombre de frames par animation et par direction. Ce choix représente un compromis entre qualité visuelle et coût en ressources (mémoire et performance).

Le processus de création reposait sur la génération de sprite sheets contenant une douzaine de frames brutes par animation. À partir de ces planches, une phase de sélection manuelle permettait d'identifier les frames les plus pertinentes en termes de continuité du mouvement.

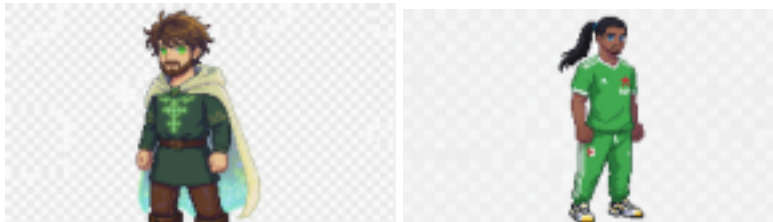
Un outil de découpage automatisé, développé spécifiquement pour ce projet, permettait ensuite d'extraire chaque frame sous forme d'image individuelle, généralement au format carré et de taille homogène.

Une fois les frames extraites, un traitement supplémentaire était appliqué afin de supprimer les arrière-plans et obtenir des sprites avec transparence (canal alpha). Pour cela, nous avons utilisé un outil externe tel que remove.bg, permettant d'automatiser efficacement cette étape. Les images finales étaient ensuite organisées de manière rigoureuse dans une arborescence de dossiers structurée par personnage, type d'animation (marche, attaque, etc.) et direction. Cette organisation facilitait grandement leur intégration dans le moteur du jeu et leur manipulation via le code.

L'évolution du visuelle des personnages:

Nous avons pu explorer plusieurs types de personnages au long de la création de ceux-ci.

Au début nous étions en conflits entre deux styles complètement différents de personnages. Nous avons donc créé des personnages différents afin de voir lequel des styles nous correspond le mieux.



Nous avons vite compris que le personnage plus mystique nous correspondait plus et nous avons suivi sur cette lancée.

C'est pour cela que nous avons changé le deuxième personnage dans cette optique



la.

Plusieurs fois...



Il nous était clair le type de visuel que chacun des personnages aurait.

Nous avons ensuite perfectionner notre idée jusqu'à arriver au résultat final de nos personnages:



Le résultat final est comme ce que l'on voulait deux personnages différents mais qui appartiennent au même monde. Un monde mystique et magique qui serait révélé par leurs accoutrements.

Realisations de Nael et Hadrien:

Packs de textures:

Le pack de texture a été un élément essentiel du graphisme car c'est ce qui permet d'avoir tout d'abord une trame pour le style et la DA du jeu en plus de faire en sorte que les maps et les graphisme sont beaucoup plus facile à importer dans notre code car tous les éléments sont à portée de mains.

Pour les créer, comme les personnages, nous avons utilisé l'IA ainsi qu'une tablette graphique sur un logiciel pour créer des dessins en pixel. Il en fallait un pour les murs, pour les sols, les obstacles ainsi que pour les différents items avec lesquels il faut interagir dans

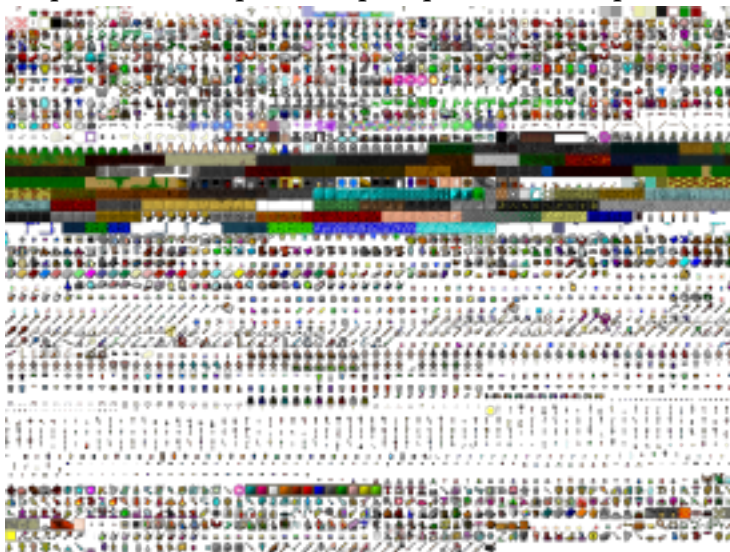
chaque salle. Ainsi, il est donc normal qu'il en aura des nouveaux pour chaque salle étant donné que les puzzles et les énigmes seront différentes mais les packs de base restés les mêmes ainsi que les salles respectent un style commun.

Evolution des packs de texture:

Comme pour les personnages ont à explorer les différentes possibilités de visuelles pour nos salles.



Jusqu'à trouver le pack complet qui nous correspond.



Map de jeu:

Pour les maps, différents modes de création ont été utilisés, l'un d'eux étant l'utilisation du logiciel tiled. En effet ce logiciel nous permet d'importer des pack de texture, crée nos maps selon nos envie, ensuite y insérer les objets avec des propriété personnalisé. Ces objets seront toutes les choses avec lesquelles nos personnage vont interagir pour sortir de la salle, ca peut etre des murs, des ponts etc..

Ensuite grâce aux objets et a des propriété personnalisé que nous leurs avons donné sur tiled, pour créer un niveau fonctionnels, il suffit d'importer la map tiled dans notre code grâce à la bibliothèque pytmx. Une fois la map importée, on utilise nos objets que l'on a créé dans tiled grâce à leurs propriétés afin que nos personnages puissent interagir avec eux jusqu'à l'ouverture de la porte.

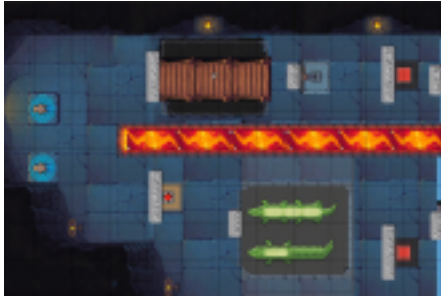
Evolution des maps du jeu:

Nous nous sommes tous tchés de faire des maps pour notre jeu et puisque nous apprenions tous comment le faire cela à résulté en plusieurs premières maps.

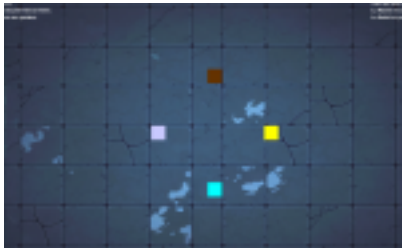
De cette sorte là nous avons tous appris d'une manière ou d'une autre à utiliser tiled, donc même si on n'a pas forcément utilisé les salles de tout le monde à la fin on a tous participé à l'apprentissage des outils.

Nos premières maps:

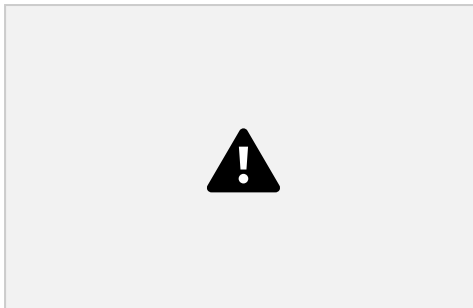
Réalisation d'Hadrien:



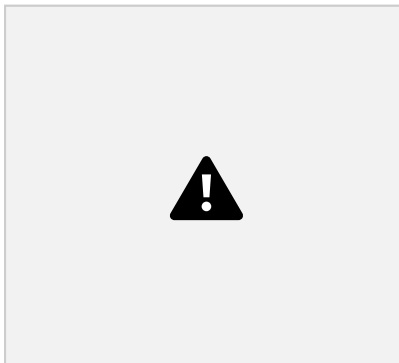
Réalisations de Julia:



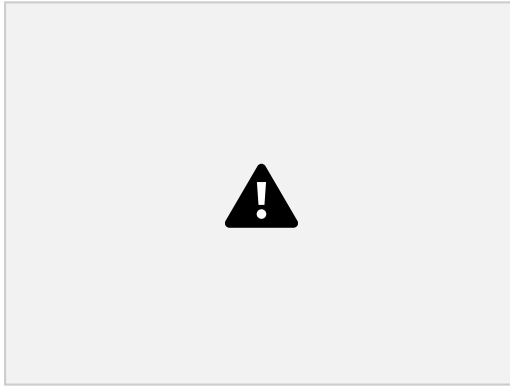
Réalisations de Ghali:



Réalisations de Melissa:



Réalisation de Nael:



On a tout exploré les packs de textures de manières différentes jusqu'à arriver à notre résultat final visible quand on joue à notre jeu.

Musique:

Pour que l'expérience du jeu soit inoubliable nous avons pensé à chaque détail en incluant la musique.

Pour cela nous avons ajouté des soundtracks à toutes les parties du jeu.

Pour améliorer l'expérience utilisateur, Nael a ensuite ajouté des effets sonores dans le menu. Grâce à cette amélioration, chaque interaction avec le menu est désormais accompagnée d'un retour sonore. Cela rend l'interface plus vivante et agréable à utiliser. Les bandes sonores utilisées pour cette partie sont donc prises de banques de sons car ce sont des sons courants.

Nous avons choisi une musique de fond pour toutes les musiques avec un style médiéval qui accompagne notre jeu.

Site Web:

Réalisation de Melissa:

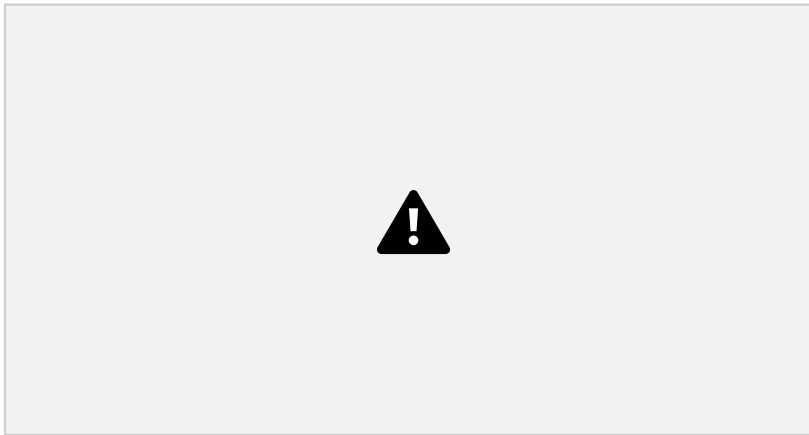
En utilisant les graphismes créés pour le jeu nous avons créé notre site web. Toutes les couleurs et images ont été choisies pour que notre site rappelle notre jeu.

Pour créer notre site, Mélissa s'est inspirée du squelette du code html d'un précédent site qu'elle avait réalisé. Elle l'a ré-adapté en fonction de nos besoins en créant les onglets Accueil, Lore, Groupe et Liens à l'aide des illustrations de Hadrien pour choisir un fond et une icône de site.

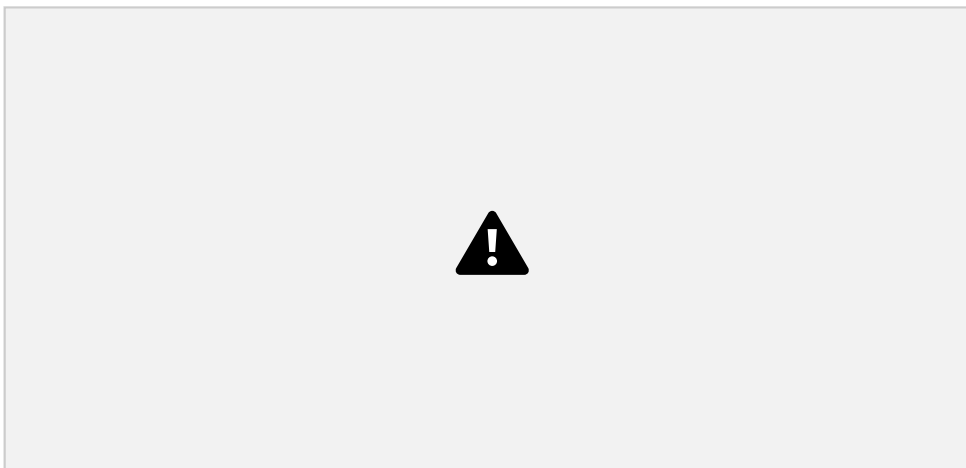
L'onglet Accueil donne un vif résumé du jeu. L'onglet Lore contient le script final de l'histoire du jeu. L'onglet Groupe présente les différents membres de l'équipe ainsi que leurs rôles respectifs. Enfin, l'onglet Liens utiles inclut la dernière version des différents livrables. Finalement, la page Accueil paraissant trop comme un article, Mélissa décida de concevoir une nouvelle page de départ du site avec moins d'éléments de texte dont le bouton lorsqu'on clique dessus, nous redirige vers les différents onglets.

Evolution du site Web:

Voici l'aperçu de la première version du site web avec comme base un site déjà créé par melissa:



Voici la version finale du site web:



Réalisation de Nael:

Menu:

Dans le cadre du développement de "What's Next?", Nael a été chargé de concevoir et d'implémenter l'interface de menu du jeu.

Le menu constitue le premier point de contact entre le joueur et le jeu : c'est lui qui donne la première impression, établit l'atmosphère et guide l'utilisateur vers les différentes fonctionnalités disponibles.

Son importance ne se limite donc pas à l'aspect purement fonctionnel — il participe pleinement à l'identité visuelle et à l'immersion globale de l'expérience proposée.

L'objectif fixé dès le départ était double : créer une interface à la fois fonctionnelle et esthétiquement cohérente avec l'univers de "What's Next?".

Pour y parvenir, Nael a utilisé le langage Python et la bibliothèque Pygame, en adoptant une approche itérative consistant à enrichir progressivement le menu par couches successives de fonctionnalités et d'améliorations visuelles.

La première version du menu était volontairement minimaliste, l'objectif étant d'abord de poser une base technique solide avant d'y intégrer des éléments plus complexes. Une fois

cette fondation établie, le menu a été enrichi de manière significative à travers plusieurs phases de développement.

Sur le plan de la navigation, un sous-menu "Jouer" a été intégré, proposant trois options distinctes : lancer une nouvelle partie, reprendre une partie existante, ou revenir au menu principal. Un menu "Paramètres" a également été développé, permettant aux joueurs d'ajuster la luminosité de l'affichage selon leurs préférences grâce à un curseur interactif.

Sur le plan technique, le menu a été configuré pour fonctionner en mode plein écran, s'adaptant automatiquement à la résolution de l'écran de l'utilisateur afin de garantir un affichage optimal quelle que soit la machine utilisée.

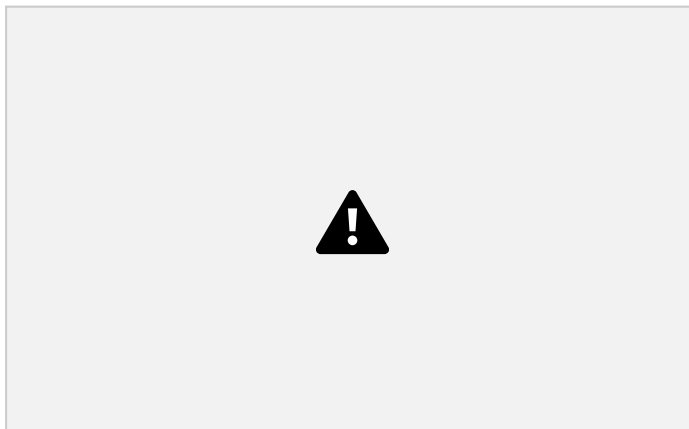
Sur le plan de l'expérience utilisateur, plusieurs améliorations ont été apportées pour rendre l'interface plus vivante et plus agréable à utiliser. Des effets sonores ont été associés aux interactions avec les boutons, distinguant les clics de navigation standard des retours en arrière.

Des effets visuels dynamiques ont également été intégrés : les boutons réagissent au survol de la souris en s'agrandissant légèrement et en s'illuminant, offrant un retour visuel immédiat qui renforce la sensation de réactivité de l'interface.

Enfin, l'élément le plus marquant du menu est l'intégration d'un fond animé sous forme de vidéo, rendu possible grâce à la bibliothèque OpenCV.

Ce fond vidéo, en boucle continue, plonge immédiatement le joueur dans l'atmosphère du jeu dès l'écran d'accueil, bien avant même que la première salle ne soit chargée.

Cette décision technique et artistique illustre la volonté de l'équipe de soigner chaque détail de l'expérience proposée, y compris dans les parties du jeu les moins directement liées au gameplay.



Nous désirons que tout produit fait par nous soit à l'image du jeu, du rendu à l'exécutable lui-même. En ayant cela comme objectif nous avons fait de notre mieux pour créer une identité visuelle à notre jeu.

Identité visuelle:

L'identité visuelle de "What's Next?" a été pensée dès le début du projet comme un élément à part entière de l'expérience de jeu, au même titre que les mécaniques de gameplay ou la narration.

Notre objectif était de créer une **cohérence graphique forte et reconnaissable**, capable de plonger immédiatement le joueur dans l'univers que nous avons imaginé.

Le choix d'une couleur dominante s'est imposé naturellement : **le vert**. Cette couleur, dans sa déclinaison émeraude et lumineuse, incarne parfaitement la dimension mystique et surnaturelle de notre univers.

Elle évoque à la fois la **magie, le danger et le mystère** — trois sensations que nous souhaitons faire ressentir au joueur tout au long de son aventure.

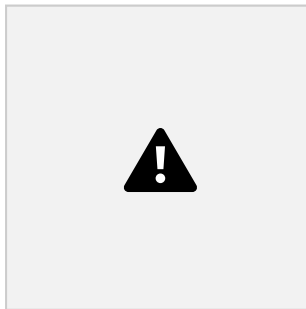
Le vert est omniprésent dans le jeu : il caractérise le cristal central de l'histoire, illumine certains éléments d'interface, et teinte l'atmosphère générale des environnements souterrains que les joueurs traversent.

Autour de cette couleur principale, nous avons construit une palette complémentaire volontairement sobre et ternie.

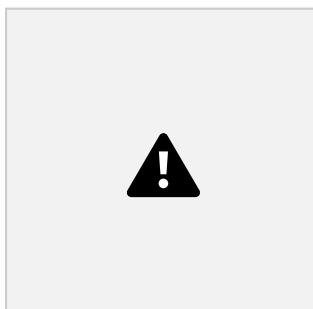
Le marron, couleur de la terre et de la pierre, renforce l'atmosphère des donjons et des ruines.

Le bleu, utilisé notamment dans la conception de l'un des deux personnages jouables, apporte une touche de contraste froid qui distingue visuellement les deux protagonistes tout en restant cohérent avec l'ensemble de la direction artistique. Deux objets iconiques ont été identifiés comme les piliers visuels de l'identité du jeu.

Le cristal vert constitue l'objet central de la narration : c'est lui qui donne leurs pouvoirs aux personnages, qui alimente leur obsession et qui guide leur progression dans les donjons. Sa représentation visuelle, lumineuse et translucide, devait immédiatement communiquer sa nature surnaturelle et précieuse.



Le chat noir, présent dès la salle tutorial, incarne quant à lui la dimension plus mystérieuse et imprévisible du jeu : animal de compagnie de la vieille dame, il devient un obstacle inattendu et contribue à l'humour discret qui ponctue l'aventure.

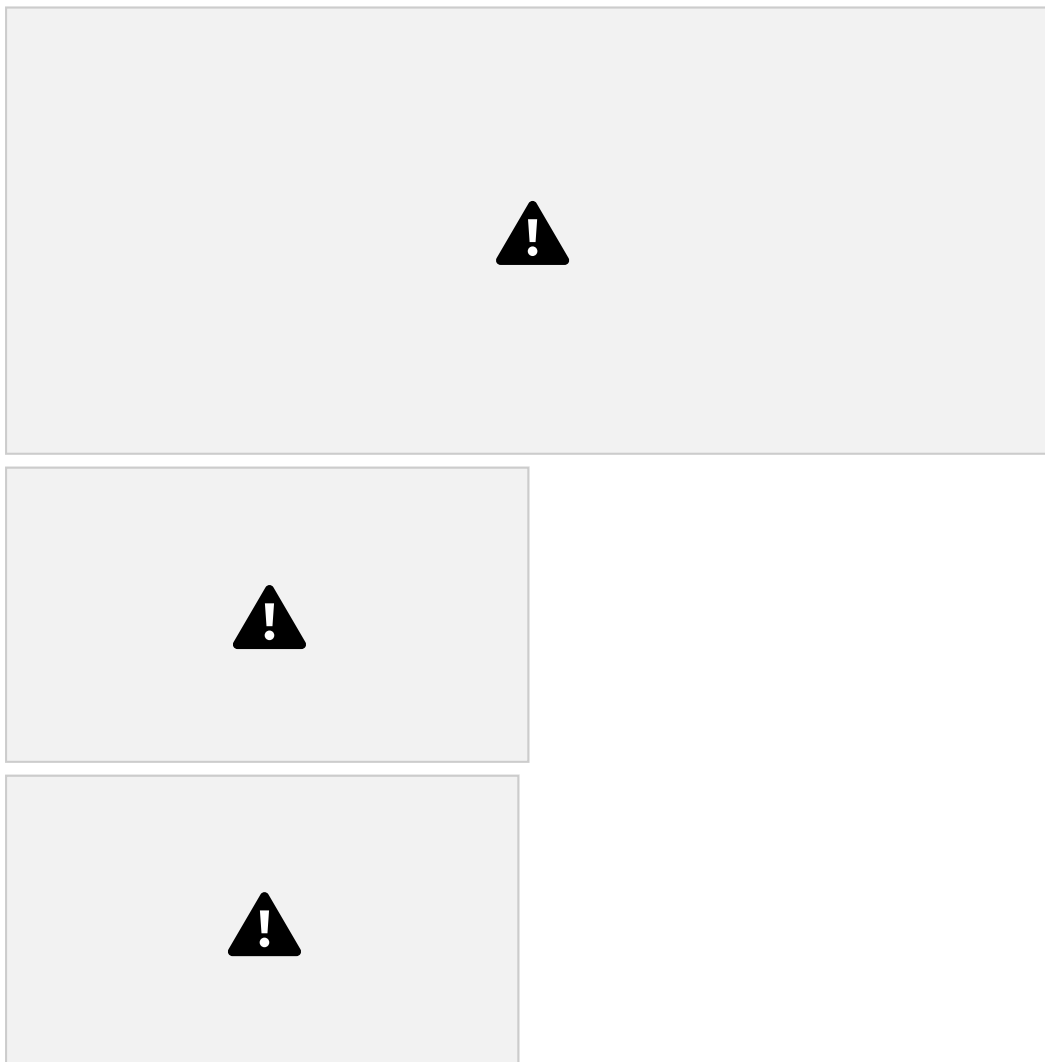


Ces deux objets ont fait l'objet d'illustrations spécifiques, réalisées pour donner une personnalité visuelle concrète au jeu et servir de références tout au long du développement.

En amont de la création des premières maps, nous avons également produit une série d'illustrations conceptuelles destinées à nous aider à visualiser collectivement l'atmosphère graphique que nous visions.

Ces illustrations ont joué un rôle fondamental dans le processus créatif : elles ont permis à chaque membre de l'équipe de partager une vision commune avant même que la première ligne de code ne soit écrite.

Elles ont ensuite servi de référence directe pour l'ensemble des productions graphiques et techniques qui ont suivi, notamment la conception du site internet, dont la palette de couleurs et l'ambiance visuelle s'en inspirent directement, le développement du menu principal, qui reprend les mêmes codes graphiques pour assurer une continuité entre l'interface et le jeu, ainsi que la création des personnages jouables, dont les couleurs et le style ont été définis en cohérence avec cette direction artistique établie dès le départ.



Au long de notre travail sur l'identité visuelle de notre projet nous avons rencontré un problème majeur concernant le travail que nous faisons lors de notre phase d'apprentissage des différents outils nécessaires à la réalisation du jeu.

Ces travaux nous ont beaucoup aidé à comprendre les outils mais ils nous ont aussi fait perdre beaucoup du temps que nous pensions utiliser autre part.

Ce phénomène n'est pas seulement rencontré lors de la partie visuelle de notre jeu mais tout au long du processus.

Travail abandonné ou non utilisé:

Au cours du projet, certaines parties du travail réalisé n'ont finalement pas été utilisées dans la version actuelle du jeu.

Par exemple, plusieurs propositions de lore rédigées par Ghali n'ont pas été retenues dans la version finale de l'histoire. Une partie du code réalisé par Julia n'est plus utilisée aujourd'hui et certaines maps créées par Hadrien ont dû être modifiées ou refaites.

Même si ces situations peuvent donner l'impression d'avoir perdu du temps, elles font également partie du processus normal de création d'un projet. Ces étapes nous ont permis de tester différentes idées et de mieux comprendre ce qui fonctionne ou non dans le jeu. Pour mieux comprendre comment nous avons réussi à passer de l'idée du jeu à l'exécution de celui-ci, il faut comprendre comment on sait organiser pour le faire.

Organisation:

L'organisation est la base d'un projet réussi mais il est très difficile de le mettre en place à plusieurs.

Organisation du temps de travail:

Lors de nos premières réunions on a estimé nécessaire la présence de tous les membres à chaque session de travail et un consensus sur les différentes mesures implémentées.

Rapidement nous avons compris que la disponibilité des membres ne coïncide pas de la sorte à toujours pouvoir organiser des séances collectives de travail.

L'organisation du travail à cinq personnes sur une période relativement courte a également été une difficulté importante.

Entre les cours, les travaux pratiques, les partiels et les rattrapages, il a été très difficile de trouver des moments où l'ensemble du groupe pouvait se réunir pour travailler ensemble. Une grande partie du travail a donc été réalisée de manière autonome par les différents membres du groupe.

Cela a parfois conduit à une situation où certaines tâches étaient réalisées rapidement à la dernière minute ou concentrées sur quelques personnes.

Nous n'avons pas encore trouvé de solution parfaite pour ce problème d'organisation, surtout que la solution dépend beaucoup du groupe, mais les difficultés rencontrées nous ont permis de mieux comprendre les contraintes du travail en groupe et d'avancer avec plus de clarté sur la manière dont nous devons communiquer et coordonner notre travail.

On a prioriser le travail en petit groupe ou en solitaire pour nous permettre d'avancer plus vite.

Grâce à nos outils de communications et développements on peut partager nos avancées au fur et à mesure qu'on les effectue.

Outils de communications:

Depuis la création du groupe il nous a semblé important de pouvoir centraliser les informations relatives au projet.

Pour communiquer entre nous, nous avons d'abord créé un groupe Snapchat. Cependant, nous avons rapidement vu que ce n'était pas très pratique pour partager des documents. Nous avons donc décidé de passer sur WhatsApp, qui permet de mieux envoyer des fichiers.

Ensuite, nous avons créé un serveur Discord afin de mieux organiser les discussions. Cela nous permet d'avoir plusieurs canaux pour les différents sujets du projet.

Aujourd'hui notre serveur discord est organisé de la sorte:

- Salons textuels:

1. General:

Ce salon nous permet de discuter des avancées générales du projet.

2. Lien:

Ce salon nous permet de mettre les liens utiles du projet.

3. Level design

Ce salon nous permet de discuter des avancées concernant le level

design. 4. Idee

Dans ce salon nous proposons de nouvelles idées.

5. Art design

Ce salon nous permet de discuter des avancées concernant le art

design. 6. Gameplay

Ce salon nous permet de discuter des avancées concernant le

gameplay. 7. Hors sujet

Dans ce salon nous partageons des informations qui ne concernent pas le projet aux membres de l'équipe.

8. Organisation

Dans ce salon nous discutons des deadlines qui nous sont imposées.

9. Prog

Ce salon nous permet de discuter des avancées concernant la

programmation. 10. Docs

Dans ce salon nous partageons des documents du projet.

- Salons vocaux:

1. Salle de reunion
2. Salle d'étude 1
3. Salle d'étude 2

Outils de développements:

Pour le développement du jeu, nous avons utilisé Visual Studio Code pour programmer et Git pour gérer le code. Nous avons également utilisé GitHub pour partager le projet et garder une sauvegarde du code.

Pour cela Julia a dû rechercher les spécificités de l'outil git notamment en utilisant ce site: - [Git - Démarrer un dépôt Git](#)

Il a aussi été nécessaire de comprendre l'outil vscode grâce à ce site:

- [Version control in VS Code](#)

Et maintenant nous avons un repository jeu partager (cf annexe 4), avec comme

lien: - <https://github.com/julug-26/jeu.git>

Visual Studio nous permet de travailler chacun de son côté tout en ayant des données centralisées sur le cloud. On utilise le système utilisé pour nos tp ce qui a été plus facile pour nous à mettre en place.

Réalisations de julia:

Rendues:

La première tâche que le groupe a dû effectuer est de **remplir le**

cdet: Le cdet (cahier des charges techniques):

Cet outil nous a permis de mieux organiser la gestion temporelle de notre projet ainsi que nos attentes fonctionnelles.

Attendus première soutenance:

En ce qui concerne le planning nous avons essayé d'espacer le plus possible les différentes tâches à effectuer tout en se focalisant dans la répartition cohérente et progressive des tâches.

Il nous semblait donc logique de commencer par l'idéalisation du jeu: écrire le lore, scripter les salles, idéaliser l'ambiance et le thème du jeu. C'est sur cela que l'on s'est concentré pour

la première soutenance.

A la suite, nous avons décidé de faire une répartition graduelle de la plus grosse partie du travail: coder les salles. On a décidé d'en faire 10 et donc d'en finir 2 pour la première soutenance, 6 pour la deuxième et 10 pour la dernière. Le menu constituant une partie importante du visuel, il nous a paru nécessaire de le rendre dès la première soutenance.

Script et lore:

Pour la première soutenance nous avons prévu de finir le script à 100% car celui-ci représentait pour nous la base de notre travail.

Avec une histoire bien définie nous pouvions mieux effectuer le reste du jeu tout en gardant une cohérence nécessaire quand on travaille en équipe.

Thèmes des pièces et script des salles:

Pour cette première phase il nous semblait important de conceptualiser notre projet c'est pour cela que nous avons décidé d'imaginer les salles à 50%.

Menu du jeu:

Le menu du jeu devait être terminé à 100 % pour la première soutenance. Le menu représente une partie importante de l'expérience utilisateur, car c'est la première chose que le joueur voit lorsqu'il lance le jeu.

Développement des salles:

Pour la première soutenance nous avons envisagé de finir à 20% les salles du

jeu. IA et réseau:

Pour la première soutenance nous avons envisagé la mise en place des systèmes d'intelligence artificielle et de réseau, qui devaient être avancés à 100 %.

Level design:

Pour cette soutenance le level design devait être fait à 50%.

Game design:

Pour cette soutenance le game design devait être fait à 50%.

Attendus deuxième soutenance:

La deuxième soutenance avait pour objectif de montrer que la majorité des éléments de conception du projet étaient finalisés. Le script, le lore, le menu, la soundtrack, le game design et le level design devaient être terminés, tandis que le développement des salles devait déjà être bien avancé.

Cette étape permettait de vérifier que le projet avançait correctement et que toutes les bases nécessaires étaient en place pour continuer le développement du jeu et préparer la soutenance finale.

Script et lore:

Pour la deuxième soutenance, le script du jeu devait être finalisé à 100 %. Cela signifiait que l'histoire principale du jeu devait être complètement définie et cohérente avec le reste du projet.

Dans notre projet What's Next ?, le lore avait déjà été commencé lors de la première soutenance. Ghali avait proposé plusieurs idées d'histoire afin que le groupe puisse choisir celle qui correspondait le mieux au type de jeu que nous voulions créer. La version finale du lore a ensuite été retravaillée et rédigée par Nael afin de donner une histoire plus claire et plus immersive.

Pour cette soutenance, il était attendu que le lore et le thème des salles soient également finalisés à 100 %. Cela signifie que chaque salle devait avoir une idée claire, une logique dans le scénario et un lien avec l'univers du jeu. Le but était d'assurer une cohérence entre l'histoire, les puzzles et l'ambiance générale du jeu.

Menu du jeu:

Le menu du jeu devait être terminé à 100 % pour la deuxième soutenance. Le menu représente une partie importante de l'expérience utilisateur, car c'est la première chose que le joueur voit lorsqu'il lance le jeu.

Lors de la première soutenance, Nael avait déjà commencé à développer le menu. Pour la deuxième soutenance, l'objectif était d'avoir un menu complet et fonctionnel.

Développement des salles:

Le développement des salles devait atteindre environ 60 % d'avancement pour cette deuxième soutenance.

Les salles représentent les niveaux du jeu dans lesquels les joueurs doivent résoudre des énigmes pour progresser. Dans notre projet, plusieurs salles ont été imaginées avec des puzzles différents qui reposent principalement sur la coopération entre les deux joueurs.

Pour cette soutenance, l'objectif était donc d'avoir plusieurs salles déjà fonctionnelles afin de montrer que le gameplay du jeu fonctionne correctement.

Soundtrack:

La soundtrack du jeu devait également être finalisée à 100 % pour la deuxième soutenance. Dans notre projet, les sons devaient être intégrés dans le menu et dans certaines interactions du jeu. L'objectif était d'avoir une ambiance sonore cohérente avec le thème du jeu et avec l'atmosphère des différentes salles.

IA et réseau:

Un autre élément attendu pour la deuxième soutenance concernait la mise en place des systèmes d'intelligence artificielle et de réseau, qui devaient être avancés à 100 %.

Level design:

Le level design devait être terminé à 100 % pour cette soutenance.

Le level design correspond à la conception des différentes salles du jeu : leur structure, la disposition des obstacles et la manière dont les puzzles sont organisés.

Dans notre projet, cela signifiait que toutes les salles prévues devaient être conceptualisées, même si certaines n'étaient pas encore complètement codées. L'objectif était d'avoir une progression logique dans la difficulté des puzzles et dans la manière dont les joueurs découvrent les mécaniques du jeu.

Game design

Enfin, le game design devait également être finalisé à 100 %.

Le game design correspond à l'ensemble des règles et des mécaniques du jeu. Cela inclut par exemple la manière dont les joueurs interagissent avec les objets, comment les puzzles fonctionnent ou encore comment la progression dans le jeu est organisée.

Dans What's Next ?, le game design repose principalement sur la coopération entre les deux joueurs. Les puzzles ont été conçus pour obliger les joueurs à communiquer et à travailler ensemble afin de progresser dans les différentes salles.

Attendue soutenance finale:

Pour la soutenance finale nous voudrions avoir fini tous les domaines à 100%.

Cela signifie avoir fait 10 salles avec au moins une contenant de l'IA en coopération donc avec un réseaux en lan, avec un thème et un scripte pour chaque salle, un soundtrack cohérent qui accompagne le jeu, un menu fonctionnelle reliée aux salles et un lore recenser dans l'ensemble du jeu.

Pour la dernière soutenance nous voulions un exécutable qui marche ou tout est relié pour créer un vrai jeu.

Pour le cddt nous devons aussi répartir les rôles:

Repartition des roles:

Lors de la répartition des rôles quelques rôles nous ont parues évidents: programmation, audio, organisation. Nous avons aussi compris la nécessité du level et game design mais avons du mal à les étiqueter, puis nous avons découverts les termes corrects. La tâche IA et réseaux nous a été soulevée par un jury, nous avons donc ajouté par la suite cette tâche.

Pour ce qui est de l'attribution des tâches à chaque membre ont a opté par une répartition centrée sur les différentes aptitudes des membres du groupe.

1. Programmation

Le pôle programmation constitue la base technique du projet. Il est chargé de l'implémentation de l'ensemble des fonctionnalités définies par les autres pôles, en traduisant les concepts et mécaniques de jeu en code opérationnel.

Ce pôle inclut notamment le développement des systèmes de jeu, la gestion des interactions, l'implémentation des mécaniques principales ainsi que l'optimisation technique et la maintenance du code.

Le responsable de ce pôle est Ghali, qui assure la coordination du développement technique et veille à la stabilité, à la cohérence et à la qualité du code tout au long du projet.

2. Level Design

Le pôle level design est un élément central de l'expérience de jeu, dans la mesure où il structure la progression du joueur et les défis qui lui sont proposés.

Il est chargé de la conception et de l'agencement des différents niveaux du jeu, en tenant compte de la progression, de l'équilibrage, du rythme et de la cohérence globale avec les mécaniques de gameplay.

Ce pôle est placé sous la responsabilité de Melissa, qui veille à proposer des niveaux cohérents, équilibrés et adaptés à l'expérience de jeu souhaitée.

3. Game Design

Le pôle game design regroupe l'ensemble des éléments conceptuels définissant le fonctionnement et l'identité du jeu.

Il comprend la définition des règles, des mécaniques de gameplay, des interactions, ainsi que l'intégration d'éléments visuels et ergonomiques facilitant la compréhension du jeu par le joueur et renforçant son attractivité.

Le responsable de ce pôle est Nael, garant de la cohérence globale du gameplay et de la qualité de l'expérience de jeu proposée.

4. Intelligence artificielle et réseaux

Ce pôle est chargé de la conception et de l'implémentation de l'intelligence artificielle du jeu, notamment le comportement des entités non-joueurs, ainsi que du développement des fonctionnalités liées au multijoueur et aux communications réseau.

Le responsable de ce pôle est Hadrien, qui veille au bon fonctionnement des systèmes d'intelligence artificielle et des interactions réseau, tout en assurant leur intégration cohérente avec les autres composantes du projet.

Ce choix de répartition nous a coûté chers, avec du recul le fait qu'initialement on est regroupé l'intelligence artificielle et le réseau dans une seule tâche n'était pas forcément la meilleure des idées.

Nous nous sommes rapidement rendu compte que ces deux domaines sont très différents et nécessitent chacun beaucoup de travail. De plus, une grande partie du fonctionnement du jeu repose sur le système de réseau, notamment pour permettre la coopération entre les joueurs.

Nous avons donc compris qu'il aurait été plus pertinent de séparer ces deux parties du projet dès le début afin de mieux répartir le travail.

Ce choix a en partie été fait après une remarque du jury sur notre répartition des rôles. Nous

n'avions pas pensé à la notion réseaux du projet. Cela a conduit forcément à des

5. Audio

Le pôle audio est responsable de la création et de l'intégration de l'environnement sonore du jeu. Il inclut la conception des musiques, des effets sonores et des éléments audio interactifs contribuant à l'immersion du joueur.

Ce pôle est également placé sous la responsabilité de Hadrien, qui veille à la cohérence artistique et à la qualité sonore du projet.

6. Organisation

Le pôle organisation assure la gestion globale du projet. Il est chargé de la planification des différentes phases de développement, du suivi de l'avancement, du respect des délais, de l'organisation des réunions et de la coordination entre les différents pôles. La responsable de ce pôle est Julia, garante du bon fonctionnement de l'équipe et de la communication interne.

Contribution de Julia:

Au sein de l'équipe Boneca Ambalabu, j'ai assumé un rôle transversal d'organisation et de coordination tout au long du projet. Bien que cette dimension du travail soit moins visible que les contributions techniques, elle a constitué une part essentielle du bon déroulement du projet.

J'ai notamment été chargée de structurer les rendus pour chacune des trois soutenances : élaboration des sommaires, mise en forme des documents, vérification de la cohérence globale des contributions de chaque membre.

J'ai également conçu et réalisé le support PowerPoint de présentation pour la soutenance finale, en veillant à ce qu'il reflète fidèlement l'ensemble du travail accompli par l'équipe.

Ce rôle de coordination a impliqué une contrainte particulièrement exigeante : pour pouvoir évaluer l'avancement de chacun, identifier les blocages, apporter de l'aide et dépanner mes coéquipiers, il m'a fallu me familiariser avec l'ensemble des composantes techniques du projet.

J'ai ainsi exploré, à des degrés divers, la programmation pygame, la création de maps dans Tiled, les outils de pixel art, le système réseau et l'intelligence artificielle — à l'exception du site internet. Cette démarche n'était pas prévue initialement, mais elle s'est imposée comme une nécessité : il est impossible de coordonner efficacement un travail qu'on ne comprend pas.

Cela a représenté un investissement personnel considérable, mais il m'a permis d'avoir une vision globale du projet et d'intervenir concrètement lorsque certaines parties prenaient du retard ou se retrouvaient sans responsable.

Sur le plan humain, j'ai régulièrement assuré un rôle de médiation au sein du groupe : gestion des tensions, motivation des membres dans les moments de découragement, suivi de l'avancement de chacun et vérification que les tâches assignées étaient bien réalisées dans les délais.

Ce rôle de coordination s'est avéré d'autant plus complexe que je suis, de mon propre aveu, la membre du groupe ayant le moins d'expérience en jeux vidéo. Cette position m'a placée dans

une situation délicate pour évaluer la complexité des tâches et les répartir efficacement — mais je l'ai assumée du mieux possible, en cherchant constamment à comprendre le travail de chacun pour mieux le coordonner.

En parallèle de mon rôle organisationnel, j'ai été amenée à contribuer directement au développement du jeu, notamment en prenant en charge des parties initialement destinées à d'autres membres du groupe.

J'ai ainsi développé la première salle du jeu, dont la conception et le code ont été réalisés intégralement par mes soins lorsque cette tâche s'est retrouvée sans responsable clairement désigné. De même, j'ai pris en charge le développement d'une salle intégrant des mécaniques d'intelligence artificielle, afin de pallier le retard pris sur cette composante du projet.

Pour la soutenance finale, j'ai également développé un système de levier interactif destiné à enrichir les mécaniques de gameplay, illustrant ma capacité à m'approprier de nouveaux concepts techniques au fil des besoins du projet.

Face aux difficultés rencontrées par l'équipe sur la question des sprites et des ressources graphiques, j'ai pris l'initiative de me former aux outils de pixel art afin de pouvoir contribuer à cet aspect du projet.

Cette démarche s'est imposée lorsqu'il est apparu que les sprites des personnages principaux ne seraient pas livrés dans les délais prévus. J'ai alors exploré plusieurs logiciels et approches — génération par intelligence artificielle, dessin sur tablette, édition pixel par pixel — dans l'objectif de produire des ressources graphiques exploitables pour le jeu.

J'ai également pris en charge la rédaction du manuel d'installation, d'utilisation et de désinstallation du jeu. Afin de garantir l'exhaustivité et la fiabilité de ce document, j'ai procédé à une installation complète du jeu sur un environnement différent de celui utilisé pendant le développement — en l'occurrence, l'ordinateur de ma sœur fonctionnant sous macOS — en passant par une clé USB comme support de transfert.

Cette démarche m'a permis de documenter précisément chaque étape du processus d'installation, d'identifier les éventuels prérequis techniques et de rédiger un manuel accessible à un utilisateur n'ayant aucune connaissance préalable du projet.

Je ne suis pas entièrement satisfaite du résultat final, car il ne correspond pas à ce que nous avions imaginé collectivement en début d'année.

Cependant, je suis convaincue d'avoir contribué du mieux possible à la réussite du projet, en m'adaptant constamment aux besoins de l'équipe et en prenant en charge des responsabilités qui dépassaient parfois le cadre de mes attributions initiales.

Cette expérience m'a appris que la gestion d'un projet de groupe ne s'improvise pas, et que la coordination humaine est tout aussi exigeante — et tout aussi indispensable — que la contribution technique.

Après l'organisation, il y a l'exécution même du projet avec ces différentes parties.

Pour chaque partie on a une explication détaillée des différents membres de l'équipe sur leurs avancées.

Programmation:

La gestion des ressources graphiques et l'intégration des personnages dans un moteur de jeu constitue l'un des aspects les plus complexes et les plus fondamentaux du développement d'un jeu vidéo en deux dimensions.

Il s'agit en effet du cœur technique du projet : sans une gestion rigoureuse des sprites, des tilesets et des chemins de fichiers, l'ensemble des mécaniques de jeu développées par l'équipe — les collisions, les transitions entre salles, les interactions coopératives — devient totalement inutilisable. C'est précisément ce défi que nous avons dû relever tout au long du projet.

Dans cette section, nous présenterons les différentes étapes qui ont conduit à l'intégration fonctionnelle des personnages et des cartes dans notre jeu.

Nous aborderons dans un premier temps la conception et la structure des spritesheets de personnages, qui définissent l'ensemble des animations de déplacement.

Nous expliquerons ensuite comment les cartes ont été créées dans Tiled Map Editor et chargées dans pygame grâce à la bibliothèque pytmx.

Enfin, nous reviendrons sur les problèmes techniques récurrents liés aux chemins de fichiers absolus et relatifs, qui ont constitué un obstacle majeur lors des transferts de fichiers entre les différentes machines de l'équipe, et sur les solutions que nous avons mises en place pour les résoudre durablement.

Première phase de développement:

Développement des salles:

Contribution de Julia:

Le développement des salles du jeu a débuté par une phase d'exploration et d'apprentissage menée par Julia Guimaraes. N'ayant pas d'expérience préalable avec la bibliothèque Pygame, Julia a commencé par se documenter de manière autonome, en consultant notamment la documentation officielle de Pygame ainsi que des tutoriels en ligne.

Cette **phase d'apprentissage** lui a permis de s'approprier les fonctions fondamentales nécessaires au développement du jeu.

Elle a ainsi maîtrisé les bases de l'initialisation et de l'arrêt du moteur Pygame, les méthodes d'affichage d'images et de textes à l'écran, ainsi que les outils de dessin de formes géométriques.

Ces premiers acquis constituent le socle technique sur lequel l'ensemble du développement des salles a été construit.

Dans un premier temps, Julia a développé une approche procédurale simple, gérant les personnages et leurs déplacements à l'aide de variables définies en dehors de toute structure orientée objet.

Cette première version, bien que fonctionnelle, présentait des limites évidentes en termes de lisibilité et de maintenabilité du code. Julia a donc fait évoluer son approche en adoptant une architecture orientée objet, créant des classes dédiées à la gestion des personnages, des obstacles et des éléments interactifs.

Cette transition a considérablement amélioré la clarté du code et facilité l'ajout de nouvelles fonctionnalités au fil du développement.

Grâce à ces outils, Julia a pu conceptualiser et développer un squelette fonctionnel pour les deux premières salles du jeu, posant ainsi les bases du système de salles qui a ensuite été repris et enrichi par l'ensemble de l'équipe.

Première salle:

La première salle du jeu avait initialement été imaginée par Hadrien, dont la vision reposait sur deux piliers fondamentaux : la gestion des obstacles et la coopération entre les deux joueurs.

Hadrien s'étant concentré sur le développement graphique du projet, c'est Julia qui a pris en charge la conception technique et la réalisation de cette salle, en s'appuyant sur les grandes lignes définies initialement.

La salle a été structurée autour de deux phases distinctes, chacune reposant sur une mécanique coopérative différente.

La première phase introduit le concept du levier : l'un des joueurs doit actionner un mécanisme qui supprime l'obstacle bloquant la progression de son coéquipier. Cette mécanique simple mais efficace oblige immédiatement les joueurs à communiquer et à coordonner leurs actions, établissant ainsi le ton coopératif de l'ensemble du jeu.

La seconde phase repose sur une action simultanée des deux joueurs : ces derniers doivent saisir le même code au même moment pour faire disparaître l'obstacle suivant. Un retour visuel clair a été intégré — les boutons passent au vert lorsque le bon code est correctement saisi — afin d'informer les joueurs de l'avancement de leur progression sans ambiguïté.

Deuxième salle:

La conception de la deuxième salle a donné lieu à un processus créatif collaboratif particulièrement riche, impliquant plusieurs membres de l'équipe avant d'aboutir à une version finale codée par Julia.

Une première version de la salle avait été imaginée par Julia elle-même. Dans cette conception initiale, les deux joueurs pénètrent dans une salle fermée où un code à trois chiffres est affiché, contenu dans des sphères verdâtres à l'aspect surnaturel. Les chiffres sont volontairement déformés par des reflets, donnant l'impression d'être lisibles sans jamais l'être totalement. Un message indique qu'un code est nécessaire pour ouvrir la porte.

Lorsque les joueurs saisissent le code apparent, celui-ci est refusé : une énigme poétique s'affiche alors, évoquant un horaire précis — 12h40 — et le symbole du soleil. En explorant la salle, les joueurs découvrent plusieurs horloges affichant des heures différentes, ainsi que des symboles célestes disposés au sol et sur les murs.

La clé de l'énigme réside dans le positionnement coordonné des deux joueurs : lorsque l'un se place près de l'horloge indiquant 12h40 et l'autre sur le bon symbole solaire, les reflets

des sphères s'alignent et le véritable code apparaît enfin. Cette version, bien qu'ambitieuse et narrativement riche, s'est avérée complexe à implémenter dans les délais disponibles.

Melissa a alors proposé une version alternative, plus directe dans sa mécanique mais tout aussi exigeante sur le plan coopératif. Cette version, finalement retenue et développée par Julia, met en scène une grande salle circulaire au centre de laquelle se dressent quatre statues anciennes, chacune associée à un symbole différent : le Soleil, la Lune, la Racine et le Cristal.

Chaque joueur perçoit des inscriptions différentes sur les murs de la salle, ce qui rend la résolution de l'énigme impossible sans communication entre les deux coéquipiers. Les inscriptions, apparemment contradictoires, doivent être croisées et analysées ensemble pour déterminer l'ordre correct d'activation des statues : Racine, puis Soleil, puis Lune.

Activer une statue dans le mauvais ordre réinitialise la progression et replace les joueurs à leur position d'origine, pénalisant toute tentative non concertée. Lorsque la bonne séquence est réalisée, les statues s'illuminent en vert une à une, offrant un retour visuel satisfaisant, et un message de validation s'affiche à l'écran.

Cette salle a cependant rencontré des difficultés techniques lors de son développement, notamment des problèmes d'affichage liés à une gestion initiale inadaptée aux résolutions en plein écran. Ces problèmes ont été partiellement corrigés mais n'ont pas pu être entièrement résolus, Julia ayant concentré ses efforts sur la finalisation de la première salle.

Deuxième phase de conception:

La deuxième phase du développement a été marquée par une refonte technique majeure du projet, menée par Nael.

Là où les premières salles avaient été développées de manière entièrement procédurale dans Pygame, Nael a introduit une nouvelle approche reposant sur l'utilisation de Tiled Map Editor pour la conception des niveaux, et de la bibliothèque pytmx pour leur intégration dans le moteur de jeu.

Cette transition a considérablement amélioré la qualité technique et visuelle du projet, tout en rendant le processus de création de nouvelles salles beaucoup plus rapide et accessible.

Contributions de nael:

Creation des maps tiled:

La première étape du travail de Nael a consisté à concevoir les salles du donjon directement dans Tiled Map Editor.

Chaque map a été organisée en plusieurs calques de tuiles superposés, chacun remplissant un rôle précis dans la construction de l'environnement.

Le calque sol contient les dalles de fond de la salle, constituant la surface sur laquelle évoluent les personnages.

Le calque wall regroupe l'ensemble des murs et obstacles solides qui délimitent l'espace de

jeu et génèrent des collisions.

Le calque deco accueille les éléments purement décoratifs — torches, statues, coffres — qui enrichissent l'atmosphère visuelle sans interférer avec le gameplay.

Enfin, le calque d'objets contient les éléments interactifs placés manuellement, tels que les items collectibles, les portes de transition et les points de spawn des joueurs.

Le tileset utilisé pour l'ensemble des maps est le DungeonCrawl ProjectUtumno Tileset, une ressource graphique open-source spécialement conçue pour les environnements de type donjon.

Il est référencé dans un fichier de configuration partagé entre toutes les maps du projet, garantissant une cohérence visuelle entre les différentes salles.

Résolution des problèmes de chemins de fichiers:

Lors du transfert des fichiers entre les différentes machines de l'équipe, un problème récurrent est apparu : les fichiers de map au format .tmx référençaient le tileset via des chemins absolus propres à la machine d'origine, rendant les maps illisibles sur tout autre ordinateur.

Ce problème, qui a bloqué le développement à plusieurs reprises, a été résolu en corrigeant manuellement les chemins dans les fichiers de configuration pour les remplacer par des chemins relatifs pointant vers le dossier assets du projet.

Cette correction, bien que simple dans son principe, a mis en lumière l'importance d'une gestion rigoureuse des chemins de fichiers dans un projet développé sur plusieurs machines simultanément.

Chargement des maps dans Pygame avec pytmx :

Pour intégrer les maps Tiled dans le moteur de jeu Pygame, Nael a développé une classe Room dédiée, reposant sur la bibliothèque pytmx.

Cette classe utilise la fonction `pytmx.load_pygame()` pour parser le fichier .tmx et en extraire l'ensemble des données de la map. Elle itère ensuite sur les calques de tuiles visibles et les affiche à l'écran tuile par tuile grâce à la méthode `surface.blit()`, reconstituant ainsi l'environnement complet de la salle en temps réel.

Cette architecture modulaire permet d'ajouter de nouvelles salles au jeu de manière simple et rapide, en créant un nouveau fichier .tmx dans Tiled et en l'associant à une nouvelle instance de la classe Room.

Création des personnages et gestion des animations :

Deux personnages jouables ont été intégrés au jeu, chacun disposant de son propre spritesheet — une image contenant l'ensemble des frames d'animation organisées en grille.

Le joueur 1 est contrôlé à l'aide des touches directionnelles, tandis que le joueur 2 utilise les touches ZQSD, permettant ainsi à deux joueurs de jouer simultanément sur le même clavier.

Chaque frame du personnage est extraite du spritesheet grâce à la méthode `subsurface()` de

Pygame, puis redimensionnée à une taille de 48 par 64 pixels pour s'adapter à l'échelle des maps.

Le déplacement gauche et droite est géré par un miroir horizontal de la sprite via la fonction `pygame.transform.flip()`, évitant ainsi d'avoir à créer des frames supplémentaires dans le spritesheet pour chaque direction.

L'animation est pilotée par un compteur de frames incrémenté à chaque déplacement du personnage.

Système de collisions :

Le système de collisions avec les murs a été entièrement automatisé à partir des données de la map Tiled.

Chaque tuile non nulle présente dans le calque wall est convertie en un objet `pygame.Rect`, constituant ainsi une liste de zones de collision générée dynamiquement au chargement de chaque salle.

Lors de chaque déplacement, la position future du joueur est comparée à l'ensemble de ces rectangles de collision. Si une intersection est détectée, le déplacement est annulé et le joueur est replacé à sa position précédente, garantissant ainsi une gestion solide et fiable des obstacles.

Système d'objets collectibles :

Un système d'items collectibles a été développé en s'appuyant sur le calque d'objets de Tiled.

Chaque objet identifié comme collectible dans la map est chargé au démarrage de la salle et stocké dans une liste d'éléments actifs.

Lorsqu'un joueur entre en contact avec l'un de ces objets, celui-ci disparaît de la salle et un message de notification s'affiche à l'écran pendant trois secondes dans un encadré centré, informant visuellement les joueurs de la collecte.

Système de portes et transitions entre salles :

Le système de transition entre les salles repose sur des objets de type porte définis dans le calque d'objets de Tiled.

La logique de transition est volontairement coopérative : pour déclencher le passage vers la salle suivante, les deux joueurs doivent se trouver simultanément sur la zone de la porte. Si un seul joueur s'y trouve, un message d'avertissement s'affiche à l'écran pour signaler à l'équipe que la condition de passage n'est pas encore remplie.

Lors de la transition, les deux joueurs sont automatiquement repositionnés à leur point de spawn respectif dans la nouvelle salle, assurant une reprise de jeu fluide et cohérente.

Gestion du projet avec Git :

Tout au long du développement, le projet a été versionné sur GitHub, permettant à l'ensemble de l'équipe de travailler de manière collaborative sur une base de code commune.

Ce versionnement s'est avéré particulièrement précieux lors de plusieurs incidents critiques — notamment la corruption d'un fichier .tmx et la suppression accidentelle d'un calque de collisions — qui ont pu être résolus rapidement en restaurant des versions antérieures stables via la commande git checkout.

Cette pratique du versionnement régulier a constitué un filet de sécurité indispensable dans un projet développé simultanément sur plusieurs machines et par plusieurs contributeurs.

Troisième phase de conception:

Conception de la Troisième Salle:

Rapport de développement - Nael:

Introduction:

La troisième salle du projet constitue une étape charnière dans la progression du jeu. Contrairement aux salles précédentes, dont la conception était relativement linéaire, cette salle a nécessité une réflexion approfondie sur la mécanique coopérative, la logique de déclenchement des événements et la gestion dynamique des calques dans l'environnement Tiled. Ce rapport retrace, dans l'ordre chronologique, l'ensemble des étapes ayant conduit à sa réalisation complète.

I. Conception initiale et création de la carte dans Tiled

1.1 Paramétrage de la carte

La conception de la troisième salle a débuté par la création d'une nouvelle carte dans Tiled Map Editor, en conservant les mêmes paramètres que les salles précédentes afin d'assurer la cohérence visuelle et technique du projet. La carte a été définie avec une orientation orthogonale, un format de données CSV, un ordre d'affichage de bas à droite, et des dimensions de 20 tuiles en largeur pour 15 tuiles en hauteur, chaque tuile mesurant 32 pixels de côté, ce qui donne une résolution totale de 640 par 480 pixels.

1.2 Import du tileset

Le tileset utilisé est le même que pour l'ensemble du projet, à savoir le fichier DungeonCrawl_ProjectUtumnoTileset.png, référencé via le fichier de définition donjon 1.tsx déjà présent dans le répertoire assets/maps/tiled/. Ce choix garantit une uniformité graphique entre les différentes salles et évite toute duplication inutile des ressources. L'import a été effectué directement depuis le panneau des jeux de tuiles de Tiled.

1.3 Organisation des calques

L'architecture des calques a été pensée pour répondre aux besoins spécifiques de cette salle, qui repose sur plusieurs mécaniques de déclenchement progressif. Les calques ont été organisés de la manière suivante, du bas vers le haut de la pile d'affichage : le calque sol contient le dallage de fond de la salle ; le calque wall regroupe l'ensemble des murs fixes qui délimitent l'espace et génèrent des collisions permanentes ; les calques wall replace 1, wall replace 2 et wall replace 3 correspondent aux murs dynamiques susceptibles de disparaître en réponse aux actions des joueurs ; le calque wall bloque, à l'inverse des précédents, est un mur initialement invisible destiné à apparaître lors d'un événement particulier ; le calque deco accueille les éléments purement ornementaux tels que les torches animées ; enfin, le calque objet, de type calque d'objets, contient l'ensemble des zones d'interaction définies manuellement.

II. Placement des éléments interactifs

2.1 Les plaques de pression

Deux plaques de pression ont été positionnées symétriquement dans la partie basse de la salle, respectivement dans le coin inférieur gauche et dans le coin inférieur droit. Chacune a été créée en tant qu'objet rectangulaire dans le calque d'objets, avec la classe plaque et les noms plaque 1 et plaque 2. Ces zones d'interaction constituent le premier déclencheur de la salle : lorsque les deux joueurs se trouvent simultanément sur une plaque chacun, cela provoque la disparition du premier mur dynamique (wall replace 1) et l'apparition du mur bloquant (wall bloque), qui sépare alors physiquement les deux joueurs dans deux espaces distincts.

2.2 Les leviers

Deux leviers ont été placés de part et d'autre de la salle, dans des zones accessibles uniquement depuis chacun des deux espaces créés par le mur bloquant. Représentés visuellement par des statues issues du tileset, ils ont été configurés en tant qu'objets avec la classe levier, nommés respectivement levier 1 et levier 2. L'activation du premier levier entraîne la disparition du calque wall replace 3, libérant ainsi un passage d'un côté de la salle, tandis que l'activation du second levier fait disparaître le calque wall replace 2, ouvrant le passage de l'autre côté. Cette mécanique force les deux joueurs à agir indépendamment mais de manière coordonnée pour progresser.

2.3 Le point de spawn et les portes

Un point de spawn unique, nommé spawn salle 3 et de classe spawn, a été placé au centre de la partie basse de la salle, entre les deux plaques de pression. Ce point sert de position d'apparition initiale pour les deux joueurs à leur arrivée dans la salle, le second joueur étant légèrement décalé sur l'axe horizontal afin d'éviter toute superposition. Deux portes ont également été intégrées : une porte en bas permettant de retourner à la salle précédente, et une porte en haut menant à la suite du jeu, toutes deux configurées avec la classe porte et une propriété personnalisée target indiquant la salle de destination.

III. Intégration dans le moteur de jeu

3.1 Chargement de la salle

La salle a été intégrée au dictionnaire des salles dans le fichier `main.py` sous la clé `salle3`, pointant vers le fichier `salle_3.tmx`. Une propriété spécifique, `both_plaques_required`, a été activée pour cette salle afin de distinguer son comportement de celui de la salle précédente, où une seule plaque suffisait à déclencher l'événement. Cette configuration permet à la classe `Room` de gérer différemment la logique de déclenchement selon la salle active.

3.2 Gestion des calques dynamiques dans `room.py`

La classe `Room` a été enrichie pour prendre en charge les nouveaux calques introduits par cette salle. Le calque `wall` bloque y est traité à rebours des autres calques dynamiques : il est initialement inactif, c'est-à-dire invisible et sans collision, et devient actif uniquement lorsque les deux plaques sont activées simultanément. Cette logique inverse a nécessité une adaptation de la méthode `check_plaques`, qui gère désormais deux comportements distincts selon la valeur du paramètre `both_plaques_required`. Les collisions sont ajoutées ou retirées dynamiquement de la liste principale à chaque frame, ce qui garantit une réponse immédiate aux actions des joueurs.

3.3 Interaction avec les leviers

La méthode `active_levier` a été conçue pour permettre à n'importe quel joueur d'activer n'importe quel levier, à condition de se trouver à proximité de celui-ci au moment de l'appui sur la touche d'interaction. La zone de détection est légèrement élargie par rapport à la zone définie dans `Tiled` grâce à la méthode `inflate`, afin de compenser les imprécisions de positionnement. Une fois activé, un levier ne peut plus être désactivé, ce qui rend les actions des joueurs irréversibles et confère à la progression un caractère définitif.

IV. Problèmes rencontrés et résolutions

La conception de cette salle a donné lieu à plusieurs difficultés techniques. La première a concerné le chemin du `tileset` dans le fichier `.tmx`, qui pointait vers un emplacement local propre à la machine d'origine et a dû être corrigé manuellement pour correspondre au chemin relatif du projet. La seconde difficulté est survenue lors des tests du calque `wall` bloque : celui-ci ne s'affichait pas et ses collisions n'étaient pas détectées, ce qui a été diagnostiqué grâce à l'ajout de `prints` de débogage dans la méthode `check_plaques`. Il s'est avéré que le fichier `Tiled` n'avait pas été sauvegardé après l'ajout du calque, rendant celui-ci absent du fichier `.tmx` lu par `pytmx`. Une simple sauvegarde a suffi à résoudre le problème. Enfin, un bug de correspondance entre les noms de touches d'interaction et les joueurs a été détecté et corrigé en inversant les assignations dans la boucle d'événements de `main.py`.

Conclusion:

La troisième salle représente la mécanique coopérative la plus élaborée du projet à ce stade. Sa conception a mobilisé l'ensemble des outils et concepts développés au fil des salles précédentes, tout en introduisant de nouvelles logiques : apparition conditionnelle d'un calque, séparation physique des joueurs, et actions irréversibles. Les difficultés rencontrées ont permis de consolider la robustesse du système de gestion des salles et ont mis en lumière l'importance de la rigueur dans la sauvegarde et la cohérence des chemins de fichiers entre les différentes machines de développement.

Contribution de ghali:

Conception de la salle tutoriel :

Ma première contribution au projet a porté sur la conception et le développement de la salle tutoriel, dont l'objectif principal était d'introduire progressivement les joueurs à l'univers et aux mécaniques de "What's Next?" avant qu'ils n'affrontent les défis des salles suivantes. Cette salle a été pensée comme un espace ouvert et lumineux, volontairement distinct de l'atmosphère sombre des donjons, afin de créer un contraste visuel marquant dès l'entrée dans le jeu.

Son level design repose sur un principe fondamental : permettre aux joueurs d'explorer librement leur environnement et d'interagir avec de nombreux objets — la cuisine, la fenêtre, la table à manger, les bibliothèques — sans que ces interactions n'aient toutes un impact direct sur la progression principale.

Ce choix délibéré vise à encourager la curiosité des joueurs et à leur faire comprendre, de manière intuitive, que l'exploration et la communication entre coéquipiers sont au cœur de l'expérience de jeu.

Sur le plan narratif, cette salle s'inscrit au tout début de l'histoire : les deux joueurs se trouvent dans l'appartement de la vieille dame qui leur a confié la mission de retrouver une relique disparue.

Le puzzle principal de cette salle consiste à accéder à la chambre principale, située en haut à droite de l'appartement, afin d'y trouver des cristaux dissimulés sous le chat de la vieille dame. Pour y parvenir, les joueurs doivent coopérer afin de faire bouger l'animal, récupérer les cristaux et quitter l'appartement pour les remettre à leur propriétaire. Ce puzzle simple mais efficace permet d'introduire la mécanique coopérative centrale du jeu sans brusquer les joueurs avec une difficulté excessive.

Conception de la salle finale :

Ma deuxième contribution majeure a été la conception de la salle finale, qui constitue l'aboutissement de l'ensemble de l'expérience de jeu et propose aux joueurs un dernier défi particulièrement intense sur le plan coopératif et émotionnel.

À ce stade de l'histoire, les deux personnages ont déjà traversé cinq salles et se trouvent enfin aux portes de la sortie du complexe souterrain. L'objectif de cette salle est de réactiver le courant électrique afin d'ouvrir la porte de sortie située en hauteur.

Pour y parvenir, les joueurs doivent localiser des câbles électriques dissimulés dans une boîte

cachée, accessible uniquement grâce à la coordination des deux joueurs : l'un doit actionner des leviers situés à l'extérieur de la salle tandis que l'autre s'y aventure.

Une fois à l'intérieur, ce dernier devra briser toutes les caisses présentes, sans y trouver les câbles — ceux-ci n'étant accessibles qu'en hauteur, hors de portée d'un seul joueur. C'est uniquement lorsque le second joueur rejoint le premier dans la salle et l'aide à s'élever jusqu'au plafond que les câbles peuvent être récupérés.

Une fois les câbles installés dans le panneau électrique et le courant rétabli, une nouvelle contrainte s'impose : la porte de sortie ne reste ouverte que très brièvement, insuffisamment pour que les deux joueurs puissent la franchir ensemble. Cette situation de crise constitue le cœur dramatique de la salle finale et donne lieu à deux fins distinctes selon les choix et les performances des joueurs.

Dans la première fin, les joueurs choisissent — ou sont contraints — de s'affronter. Le perdant sacrifie sa liberté en maintenant la porte ouverte suffisamment longtemps pour que le vainqueur puisse s'échapper.

Dans la seconde fin, les joueurs optent pour une solution coopérative : ils récupèrent des morceaux de bois dans une salle adjacente pour bloquer mécaniquement la porte. Le joueur 1 actionne le levier d'ouverture tandis que le joueur 2 positionne les morceaux de bois et son propre corps en guise de butée, déclenchant ainsi deux mini-jeux simultanés — le premier joueur doit courir en cliquant rapidement, le second doit maintenir la pression sur la porte en faisant de même.

Si les deux mini-jeux sont réussis du premier coup, les deux joueurs s'échappent ensemble et le jeu se conclut par un retour au menu principal.

En cas d'échec, les morceaux de bois se brisent et ne peuvent plus être réutilisés, forçant irrémédiablement les joueurs à s'affronter pour déterminer lequel d'entre eux pourra quitter la cave.

Cette salle finale illustre parfaitement la philosophie de "What's Next?" : chaque choix a des conséquences, la coopération est toujours possible mais jamais garantie, et l'issue du jeu dépend autant de la communication entre les joueurs que de leurs réflexes individuels."

Contribution de melissa:

Au cours du projet, j'ai été chargé du développement et de la maintenance du site internet officiel de "What's Next?", qui constitue la vitrine numérique du projet à destination des évaluateurs et du public extérieur.

Dès les premières phases du développement, j'ai mis en place l'architecture générale du site en créant l'ensemble des pages requises par le cahier des charges de l'EPITA : présentation du projet et de son univers, présentation des membres de l'équipe, ressources utilisées et documents de suivi. Cette première version du site avait pour objectif principal de répondre aux exigences formelles imposées par l'encadrement pédagogique.

À l'approche de la soutenance finale, j'ai entrepris une refonte complète du site afin d'en améliorer significativement l'aspect visuel et l'ergonomie. Cette refonte a notamment inclus l'intégration d'une page d'accueil immersive avec écran d'introduction, une réorganisation complète des contenus, l'ajout des documents manquants, l'amélioration de la navigation entre les pages ainsi que la mise en place d'une identité graphique cohérente avec l'univers

du jeu.

Le design du site s'inspire directement de l'ambiance de "What's Next?" : la palette de couleurs, dominée par des teintes sombres et des nuances de vert émeraude rappelant le cristal central de l'histoire, vise à proposer une interface moderne et immersive permettant aux visiteurs de découvrir facilement le projet et son évolution au fil des soutenances.

Parallèlement au développement du site internet, j'ai participé activement à la conception des niveaux du jeu. Ce travail a consisté dans un premier temps à imaginer la structure générale des différentes salles et des donjons, en définissant leur progression narrative et leur niveau de difficulté croissant.

J'ai ensuite réfléchi à l'emplacement des énigmes, des obstacles et des zones d'exploration au sein de chaque salle, en veillant à ce que chaque élément soit pensé pour encourager et nécessiter la coopération entre les deux joueurs. Pour mener à bien cette tâche, j'ai effectué des recherches approfondies sur les principes fondamentaux du level design, notamment sur la manière dont l'environnement peut guider le joueur, créer de la tension et favoriser les interactions coopératives.

Enfin, j'ai participé aux réflexions collectives concernant la cohérence globale du jeu, en veillant à ce que les mécaniques de gameplay, le scénario et les environnements traversés par les joueurs forment un ensemble narratif et ludique homogène, en accord avec l'identité visuelle et le lore de "What's Next?".

La mise en place d'un moteur de jeu fonctionnel, d'une gestion rigoureuse des maps et d'un système de salles cohérent a constitué le socle indispensable sur lequel repose l'ensemble du projet.

Cependant, si ces fondations techniques permettent au jeu de fonctionner, elles ne suffisent pas à elles seules à créer une expérience de jeu véritablement stimulante et immersive. Pour que les joueurs ressentent une réelle tension dans les donjons et soient poussés à coopérer face à une menace commune, il était nécessaire de donner vie à des ennemis capables de se comporter de manière autonome, intelligente et adaptative.

C'est précisément cette problématique qui a guidé le développement de la composante intelligence artificielle du projet, qui représente sans doute le défi algorithmique le plus ambitieux que l'équipe ait eu à relever.

IA:

L'intelligence artificielle des ennemis constitue sans doute l'aspect le plus ambitieux et le plus exigeant sur le plan algorithmique de l'ensemble du projet. Là où la gestion des salles ou des sprites relève principalement de la maîtrise d'outils et de bibliothèques existantes, l'implémentation d'une IA requiert une véritable réflexion algorithmique : il ne s'agit plus seulement de faire fonctionner un programme, mais de concevoir un comportement cohérent, réactif et crédible pour des entités autonomes évoluant dans un environnement dynamique. C'est précisément ce qui en fait le cœur algorithmique du projet, et ce qui en a fait l'un des défis les plus formateurs pour l'équipe.

Dans cette section, nous présenterons la démarche suivie pour concevoir et implémenter le

système d'intelligence artificielle des ennemis. Nous expliquerons dans un premier temps le choix de l'algorithme de pathfinding A*, en détaillant son fonctionnement et les raisons qui ont motivé son adoption dans le cadre de ce projet. Nous décrirons ensuite la machine à états qui structure le comportement de chaque ennemi, avec ses trois états fondamentaux que sont la patrouille, la poursuite et la fuite. Enfin, nous présenterons les deux types d'ennemis implémentés — l'ennemi de mêlée et l'archer — afin d'illustrer la flexibilité et les possibilités offertes par le système mis en place.

Première phase de développement:

Avancés de Julia:

Pour cette soutenance, j'ai été chargée de développer un système d'intelligence artificielle pour les ennemis du jeu. Ce travail a représenté un vrai défi car je ne savais pas par où commencer. En cherchant des ressources en ligne, j'ai rapidement remarqué que beaucoup de sites restaient vagues sur la manière concrète d'implémenter une IA dans un jeu en Python. J'ai donc demandé à un collègue ce qu'il utilisait dans son propre projet, et c'est lui qui m'a orienté vers l'algorithme A*, en précisant que c'était particulièrement adapté aux maps créées avec Tiled.

À partir de là, j'ai décidé de réaliser une mini-simulation indépendante en Python avec la bibliothèque Pygame plutôt que d'intégrer directement l'IA dans le jeu principal. Ce choix s'est imposé naturellement : le développement de la première salle, qui ne contient pas encore d'IA, était toujours en cours. Plutôt que d'attendre son achèvement pour commencer à travailler sur la deuxième salle, j'ai préféré avancer en parallèle sur une simulation autonome. Cela m'a permis de développer et de tester le comportement des ennemis dans un environnement simple et contrôlé, avec l'objectif d'adapter ce code aux vraies salles du jeu par la suite.

La machine à états — une progression par étapes:

Au départ, ma première version de la simulation utilisait la map Tiled déjà créée par Hadrien. Cependant, j'ai rapidement réalisé que cette map était trop grande et trop complexe pour tester une IA : les chemins étaient difficiles à contrôler et cela ralentissait inutilement le développement. J'ai donc décidé de créer un petit labyrinthe directement dans Pygame, sous forme d'une grille 2D codée en dur. Cette décision m'a permis de contrôler précisément la disposition des murs et des espaces libres, ce qui était essentiel pour tester le pathfinding.

Dans cette première version, il n'y avait qu'une seule IA qui se contentait de poursuivre le joueur et de s'arrêter sur lui au contact. Le comportement était fonctionnel mais très basique : l'ennemi n'avait aucun autre état, il suivait simplement le joueur le plus proche en permanence.

En remarquant que la simulation impliquait deux joueurs, j'ai ensuite ajouté un deuxième ennemi. C'est là que j'ai rencontré un nouveau problème : les deux ennemis ne suivaient qu'un seul joueur à la fois, et la taille de leur zone de détection était trop grande, ce qui rendait le jeu injouable. J'ai dû ajuster ces paramètres mais je n'ai pas encore trouvé une solution totalement satisfaisante — c'est un point que je devrai approfondir pour la prochaine soutenance.

C'est à ce moment-là que Nael a rédigé la charte de comportement de l'IA, qui décrit

formellement les différents états possibles d'un ennemi. Cette charte a été un vrai déclic pour moi : elle m'a permis de formaliser ce que je faisais de manière intuitive et de comprendre clairement ce que je devais implémenter. J'ai alors structuré le comportement de chaque ennemi autour de trois états distincts :

- Patrouille : lorsqu'aucun joueur n'est détecté à proximité, l'ennemi se déplace automatiquement entre plusieurs points prédéfinis sur la map. Il marque également une courte pause entre chaque point pour rendre son comportement plus naturel.
- Poursuite : dès qu'un joueur entre dans la zone de détection de l'ennemi, celui-ci quitte sa patrouille et se dirige vers le joueur le plus proche en utilisant A*.
- Fuite : si les points de vie de l'ennemi descendent en dessous d'un certain seuil, il interrompt le combat et retourne vers sa position d'origine.

Ces trois états sont gérés par un système de priorités clair : la fuite est toujours prioritaire sur tout, puis vient la poursuite, et enfin la patrouille par défaut.

L'algorithme A — fonctionnement et implémentation:

Pour que les ennemis puissent se déplacer intelligemment en évitant les obstacles, j'ai implémenté l'algorithme de pathfinding A, également appelé A-star.

Cet algorithme calcule le chemin le plus court entre deux points sur une grille en tenant compte des murs et des obstacles présents dans l'environnement. Son fonctionnement repose sur quatre composantes principales qui travaillent conjointement.

La première composante est la fonction heuristique. Son rôle est de calculer une estimation de la distance entre deux cases de la grille. Nous utilisons pour cela la distance de Manhattan, c'est-à-dire la somme des différences absolues en x et en y entre deux positions.

Ce choix est justifié par le fait que nos ennemis ne se déplacent qu'en quatre directions — haut, bas, gauche et droite — sans possibilité de mouvement en diagonale. La distance de Manhattan est donc la mesure la plus précise et la plus efficace dans ce contexte.

La deuxième composante est l'initialisation de l'algorithme. Trois structures de données sont créées au démarrage.

L'`open_set` est une liste de cases à explorer, triée automatiquement par priorité grâce à la bibliothèque `heapq`, de sorte que la case présentant le score le plus bas est toujours traitée en premier.

Le dictionnaire `came_from` mémorise, pour chaque case visitée, par quelle case on y est arrivé : c'est cette structure qui permet de reconstituer le chemin complet une fois la cible atteinte, en remontant la chaîne de cases depuis l'arrivée jusqu'au départ.

Enfin, les dictionnaires `g_score` et `f_score` stockent respectivement le coût réel pour atteindre chaque case depuis le point de départ, et le coût estimé total qui combine ce coût réel et la valeur heuristique. C'est ce dernier score qui guide A vers la cible de manière efficace.

La troisième composante est la boucle principale de l'algorithme. À chaque itération, la case présentant le `f_score` le plus bas est extraite de l'`open_set` grâce à la fonction `heapq.heappop`.

Si cette case correspond à la cible recherchée, l'algorithme reconstitue alors le chemin complet en remontant le dictionnaire `came_from` depuis la cible jusqu'au point de départ, puis l'inverse pour l'obtenir dans le bon sens.

La quatrième composante est l'exploration des cases voisines. Pour chaque case courante, les quatre cases adjacentes sont examinées. Chaque voisin est d'abord vérifié pour s'assurer qu'il se trouve bien dans les limites de la grille, puis pour déterminer s'il s'agit d'un mur — auquel cas il est ignoré.

Si le voisin est accessible, le coût pour l'atteindre depuis la case courante est calculé en ajoutant 1 au coût actuel, chaque déplacement ayant un coût unitaire.

Si ce nouveau coût est inférieur au coût précédemment enregistré pour ce voisin, ses scores sont mis à jour et il est ajouté à l'`open_set` pour être exploré lors d'une itération ultérieure.

Enfin, une optimisation importante a été introduite pour préserver les performances du jeu : recalculer le chemin à chaque frame serait beaucoup trop coûteux en ressources processeur.

Un timer a donc été mis en place pour que le chemin ne soit recalculé que toutes les trente frames environ, soit deux fois par seconde à soixante images par seconde. Cette fréquence s'est révélée suffisante pour que le comportement des ennemis reste réactif et crédible, sans impacter les performances globales du jeu.

Les deux types d'ennemis:

Une fois la machine à états et le pathfinding fonctionnels, j'ai voulu rendre la simulation plus intéressante en implémentant deux types d'ennemis distincts, inspirés de la charte rédigée par Nael. L'idée était de montrer que le même système de base peut donner des comportements très différents selon les paramètres et les actions de chaque ennemi.

L'ennemi de mêlée (rectangle rouge) adopte un comportement agressif et direct. Dès qu'il détecte un joueur dans sa zone de détection, il calcule via A* le chemin le plus court pour l'atteindre et fonce vers lui. Lorsqu'il arrive au contact, il repousse le joueur avec un effet de knockback — un recul calculé dans la direction opposée à l'ennemi — et déclenche un flash rouge temporaire sur le joueur touché pour signaler visuellement le coup reçu. Une période d'invincibilité temporaire empêche l'ennemi d'infliger des dégâts en continu à chaque frame.

L'ennemi archer (rectangle violet) adopte un comportement plus tactique. Au lieu de foncer sur le joueur, il cherche à maintenir une distance optimale pour tirer des projectiles visibles qui se déplacent à travers la map. Si le joueur est dans sa portée de tir, l'archer s'arrête et tire. Si le joueur est trop loin, il utilise A* pour se rapprocher. Un système de cooldown entre chaque tir l'empêche de tirer en continu — il doit attendre environ une seconde entre chaque projectile. Les projectiles disparaissent automatiquement s'ils touchent un mur ou après un certain temps.

Ce qui est intéressant avec ces deux ennemis, c'est qu'ils partagent exactement le même système de machine à états et le même algorithme A*, mais leur comportement en combat est totalement différent. Cela montre la flexibilité du système mis en place et donne un aperçu de ce qui pourra être développé dans les vraies salles du jeu.

La simulation finale:

Pour rendre la démonstration concrète et présentable lors de la soutenance, j'ai intégré tous ces éléments dans une simulation jouable en plein écran. La map est un couloir avec des obstacles générés automatiquement en fonction de la résolution de l'écran, ce qui permet à la simulation de s'adapter à n'importe quel ordinateur sans modification du code.

Les deux joueurs partent du côté gauche de la map — le joueur 1 se contrôle avec ZQSD et le joueur 2 avec les flèches directionnelles. L'objectif est d'atteindre la sortie marquée EXIT à droite de la map, tout en évitant les deux ennemis qui patrouillent dans leurs zones respectives et attaquent dès qu'un joueur s'approche. Une fois les deux joueurs sortis, un écran de victoire s'affiche avec la possibilité de rejouer en appuyant sur R. La touche Échap permet de quitter à tout moment.

Pour la suite du projet, ce code devra être adapté pour fonctionner avec les vraies salles du jeu. Plusieurs points restent à améliorer : la gestion de plusieurs ennemis qui se coordonnent entre eux, l'intégration avec le système réseau pour que chaque joueur soit sur un ordinateur différent, et le remplacement des rectangles de couleur par les vrais sprites graphiques créés par Hadrien.

Deuxième phase de développement:

La deuxième phase du développement de l'intelligence artificielle a porté sur la conception d'un système de combat de type boss fight, développé par Ghali.

L'objectif de ce système était de proposer aux joueurs un défi d'un genre radicalement différent de celui des ennemis classiques : plutôt que de fuir ou d'éviter des adversaires autonomes, les deux joueurs doivent ici coordonner activement leurs attaques pour vaincre un adversaire particulièrement résistant.

Avances de ghali:

Il convient cependant de souligner que ce système de combat, bien qu'intéressant dans sa conception, ne s'inscrit pas dans le cadre défini par la charte de comportement de l'IA rédigée en amont par l'équipe.

Cette charte décrivait formellement un système d'ennemis autonomes reposant sur une machine à états composée de trois comportements distincts — la patrouille, la poursuite et la fuite — pilotés par l'algorithme de pathfinding A.

Le système de boss fight développé par Ghali repose quant à lui sur une logique de combat au tour par tour, fondamentalement différente de cette architecture. Par ailleurs, ce type de mécanique s'éloigne du genre que nous avons défini pour notre jeu : "What's Next?" a été conçu comme un puzzle coopératif, et non comme un jeu de combat ou un RPG à système de tours.

L'intégration de ce système constitue donc un écart par rapport à la vision initiale du projet, qui aurait mérité d'être discuté et validé collectivement avant sa mise en œuvre.

Ce système s'inspire directement des mécaniques de combat au tour par tour présentes dans des jeux de référence tels que Baldur's Gate ou Clair Obscur : Expedition 33.

Le principe repose sur une alternance entre phases d'attaque et phases de défense : les

joueurs doivent attaquer l'ennemi, puis se préparer à encaisser sa riposte avant de reprendre l'offensive.

Cette structure impose une communication constante entre les deux joueurs, qui doivent décider ensemble du moment opportun pour attaquer, de la répartition des rôles et de la manière de combiner leurs capacités pour maximiser les dégâts infligés tout en minimisant ceux subis.

Ce système de boss fight a été intégré à deux reprises dans le jeu, dans des contextes narratifs très distincts.

La première occurrence constitue un Easter egg dissimulé dans la salle tutorial : si les joueurs tardent à quitter l'appartement de la vieille dame après avoir récupéré les cristaux, le chat — qui avait été distrait pour permettre leur collecte — retourne à son nid, constate la disparition des cristaux et déclenche alors un combat inattendu contre les joueurs.

Cette séquence, volontairement surprenante et humoristique, récompense les joueurs curieux tout en introduisant les mécaniques de combat dans un contexte détendu. La seconde occurrence intervient dans la salle finale, lors de l'affrontement entre les deux joueurs lorsque ceux-ci ne parviennent pas à trouver une solution coopérative pour franchir la porte de sortie ensemble.

Dans ce cas, le système de combat est utilisé non plus contre un ennemi extérieur, mais entre les deux protagonistes eux-mêmes, donnant lieu à l'une des deux fins possibles du jeu.

Contribution de Melissa:

Bien que le développement principal de l'IA ait été assuré par Julia, j'ai contribué à son intégration concrète dans le jeu en créant la quatrième et dernière salle de What's Next?, qui constitue le niveau final du jeu et met en scène les ennemis intelligents au cœur du gameplay.

La salle 4 a été conçue dans Tiled Map Editor selon la même logique que les salles précédentes : organisation en calques superposés (sol, murs, décoration, objets interactifs), utilisation du tileset partagé du projet, et définition des points de spawn, des leviers et des zones de collision via des objets Tiled avec propriétés personnalisées.

Sur le plan du gameplay, la salle s'organise en deux phases distinctes. Dans un premier temps, les deux joueurs doivent activer deux leviers situés à des endroits différents de la salle tout en affrontant deux ennemis de mêlée qui ciblent chacun un joueur différent.

Une fois les deux leviers activés, un boss de fin apparaît — un ennemi de type BossEnemy, plus résistant, capable de se déplacer vers les joueurs et de tirer des projectiles. Lorsque le boss est vaincu, une séquence de générique de fin se déclenche automatiquement.

Ce générique a été implémenté dans le code principal du jeu. Il affiche un écran noir sur lequel défile verticalement la liste des membres de l'équipe, précédée d'un message de remerciement au joueur.

Le défilement est fluide et temporisé, avec la possibilité de quitter à tout moment via une touche. Cette séquence donne au jeu une conclusion soignée et marque l'aboutissement de plusieurs mois de travail collectif.

Si l'intelligence artificielle des ennemis représente le défi algorithmique majeur du projet, elle ne constitue pas pour autant l'unique complexité technique à laquelle l'équipe a dû faire face.

En effet, le cahier des charges imposait dès le départ une contrainte fondamentale : le jeu devait fonctionner en coopération locale sur réseau, ce qui impliquait de transformer un prototype mono-écran en une véritable application multijoueur synchronisée.

C'est précisément cette problématique réseau qui représente la troisième et dernière grande composante technique du projet.

Réseaux:

La mise en place d'une architecture réseau fiable et performante constitue l'un des défis les plus exigeants du développement de jeux vidéo, car elle impose de concilier deux impératifs souvent contradictoires : la réactivité en temps réel et la cohérence des données entre les machines.

Dans le cadre de ce projet, l'enjeu était d'autant plus complexe que la couche réseau devait s'intégrer sans rupture dans un code existant — le moteur de rendu Pygame, la gestion des cartes Tiled, les animations de personnages — sans en altérer le fonctionnement."

Dans cette section, nous présenterons l'architecture réseau mise en place pour permettre la coopération en LAN entre deux joueurs.

Nous expliquerons dans un premier temps le choix d'un serveur TCP autoritaire comme source unique de vérité pour la synchronisation des états de jeu.

Nous détaillerons ensuite les mécanismes de communication bidirectionnelle entre le serveur et les clients, notamment la gestion des threads, du buffer circulaire et du throttling réseau.

Enfin, nous reviendrons sur les principaux obstacles techniques rencontrés lors du développement et les solutions apportées pour garantir une synchronisation stable et performante.

Première phase de développement:

Avancés de Hadrien :

Dans le cadre du projet SAE J3D "What's Next?" à EPITA, le développement de la composante réseau a constitué une étape stratégique et particulièrement exigeante, dont l'objectif principal était de transformer un prototype solo fonctionnel en une application

multijoueur coopérative respectant scrupuleusement les spécifications du cahier des charges.

La contrainte fondamentale de ce développement était de préserver intégralement le code existant — incluant le moteur de rendu Pygame, la classe Player gérant les animations de personnages en quatre directions, et la classe Room basée sur pytmx pour le chargement des cartes Tiled — tout en y ajoutant une couche réseau transparente assurant une synchronisation parfaite entre les deux joueurs.

La première phase du développement a porté sur la conception et l'implémentation du serveur TCP autoritaire.

Ce serveur écoute sur l'adresse 0.0.0.0:5555, ce qui lui permet d'accepter des connexions depuis toutes les interfaces réseau de la machine hôte, qu'il s'agisse d'une connexion WiFi ou Ethernet. L'option socket SO_REUSEADDR a été activée dès le départ afin de contourner les délais d'indisponibilité du port lors des redémarrages fréquents en phase de test.

Chaque client connecté bénéficie d'un thread de gestion dédié, créé via `threading.Thread`, garantissant un traitement non-bloquant et simultané des données des deux joueurs.

Au centre de l'architecture se trouve la structure de données `game_state`, implémentée sous la forme d'un dictionnaire Python centralisé constituant la source unique de vérité pour l'ensemble du système. Cette structure stocke en permanence les positions absolues des deux joueurs, leur direction d'animation, l'identifiant de la salle courante, ainsi que l'état de l'ensemble des mécanismes de jeu, notamment les puzzles de statues, les interrupteurs et les objets collectibles.

Lorsqu'un client transmet ses inputs au serveur sous la forme d'un message JSON compact, le thread correspondant met à jour l'état officiel puis déclenche immédiatement un broadcast vers tous les clients connectés.

Cette approche dite d'autoritative server élimine tout risque de désynchronisation entre les deux machines et constitue la norme industrielle pour les jeux multijoueurs en réseau.

L'intégration de la couche réseau côté client a nécessité une refonte minutieuse de la boucle principale Pygame afin de maintenir une cadence de soixante images par seconde tout en gérant simultanément la communication bidirectionnelle avec le serveur.

La réception des mises à jour serveur s'effectue dans un thread daemon asynchrone dont la logique repose sur un buffer circulaire utilisant le caractère de saut de ligne comme délimiteur.

Cette technique résout de manière élégante les problèmes de fragmentation TCP, où un message JSON peut être reçu en plusieurs paquets discontinus : le buffer accumule les données brutes jusqu'à reconstitution complète du message, puis effectue son analyse syntaxique via la fonction `json.loads()`.

La transmission des inputs joueur intègre un mécanisme de throttling temporel fixé à cinquante millisecondes, correspondant à une fréquence réseau de vingt images par seconde.

Ce choix réduit la charge réseau de soixante-six pour cent par rapport à une synchronisation calquée sur la fréquence d'affichage, sans impact perceptible sur la réactivité du jeu. La classe Player existante a été étendue d'une méthode `get_input()` qui convertit les entrées clavier en dictionnaire JSON prêt à l'envoi, tandis que la méthode de rendu `draw()` utilise désormais exclusivement les coordonnées officielles extraites de l'état de jeu centralisé.

La classe Room est quant à elle restée totalement inchangée, assurant une parfaite continuité avec les assets Tiled développés par le reste de l'équipe.

Le processus de validation a révélé plusieurs obstacles techniques majeurs, tous résolus au fil des itérations de développement. Initialement, le serveur se terminait prématurément après l'acceptation des deux clients, provoquant des exceptions de type `BrokenPipeError` lors des tentatives d'envoi suivantes.

Cette anomalie a été corrigée par l'ajout d'une boucle infinie de maintien d'activité après la phase d'initialisation des connexions. Par ailleurs, l'utilisation simultanée de plusieurs fenêtres Pygame générait des plantages systématiques lors du changement de focus, causés par la propagation croisée des événements de fermeture entre les fenêtres.

Une gestion individualisée des événements par fenêtre, combinée à une fermeture propre des sockets dans un bloc `finally`, a permis de résoudre cette instabilité. Un autre problème critique concernait un spam réseau excessif générant jusqu'à trois mille messages par seconde, saturant à la fois la console et la bande passante disponible.

L'introduction du `throttling`, associée à une gestion silencieuse des exceptions dans le module réseau, a ramené la charge à un niveau optimal. Enfin, des processus persistants bloquant le port 5555 après une fermeture brutale des terminaux ont nécessité la mise en place d'une procédure standard de nettoyage, exécutée systématiquement avant chaque relance du serveur.

Les tests finaux, réalisés en configuration multi-fenêtres côte à côte sur une même machine, ont validé une synchronisation parfaite entre les deux instances du jeu : les déplacements effectués sur l'une se répercutent instantanément sur l'autre avec une latence inférieure à trois millisecondes.

Le système démontre une robustesse satisfaisante face aux changements de focus, aux fermetures intempestives et aux tentatives de reconnexion.

Le passage vers une configuration réseau local réelle s'avère par ailleurs trivial, nécessitant uniquement la substitution de l'adresse IP dans le fichier de configuration client.

L'architecture développée offre une scalabilité native pour l'ensemble des salles prévues au cahier des charges : l'ajout d'une nouvelle salle nécessite simplement l'extension du champ `current_room` dans le dictionnaire `game_state`, tandis que les mécanismes de puzzles et de collecte d'objets s'intègrent via une validation serveur effectuée dans la fonction de gestion des clients avant broadcast.

Ce système réseau, représentant environ huit cents lignes de code Python optimisé, permet à "What's Next?" de satisfaire pleinement les exigences multijoueurs imposées par la formation tout en préservant l'intégralité du travail graphique et de gameplay réalisé par le reste de l'équipe.

Deuxième phase de développement:

Contribution melissa:

La composante réseau du projet a été développée en collaboration avec Hadrien. Dès la deuxième soutenance, Hadrien avait posé les bases de l'architecture : un serveur TCP autoritaire en Python utilisant les modules `socket` et `threading`, capable de gérer deux clients

simultanément, de synchroniser leurs positions via des messages JSON délimités par des sauts de ligne, et de diffuser l'état du jeu à 30 fps. Ce système fonctionnait initialement en local sur une même machine.

J'ai pris en charge la finalisation de ce système pour le rendre pleinement opérationnel en configuration LAN réelle. J'ai notamment veillé à ce que le serveur écoute correctement sur 0.0.0.0 afin d'être accessible depuis toutes les interfaces réseau de la machine hôte, permettant à deux joueurs sur le même réseau Wi-Fi ou Ethernet de jouer ensemble sans configuration supplémentaire.

J'ai également travaillé sur la robustesse du système : meilleure gestion des déconnexions, stabilisation du buffer de réception pour éviter les problèmes de fragmentation TCP, et nettoyage des ressources en cas de fermeture brutale.

Enfin, j'ai intégré le mode réseau directement dans l'interface du jeu en mettant à jour le menu principal pour proposer les options « Héberger une partie » et « Rejoindre une partie » avec saisie de l'adresse IP, rendant ainsi le multijoueur accessible sans passer par la ligne de commande.

Analyse des difficultés rencontrées pendant le projet:

La réalisation de "What's Next?" nous a confrontés à des obstacles concrets qui ont significativement impacté le rendu final.

Il nous semble essentiel, dans le cadre de cette soutenance, de les analyser avec honnêteté afin d'en tirer des enseignements durables.

Le premier constat est celui d'un manque de familiarisation précoce avec les outils et les technologies du projet. Pygame, Tiled, pytmx, les systèmes de réseau TCP, l'algorithme A — aucun de ces outils n'était maîtrisé en début d'année.

Or, au lieu de prendre le temps de les explorer collectivement dès les premières semaines, chaque membre a tenté de les appréhender de son côté, souvent tardivement. Cette absence de phase d'apprentissage partagée a généré des incompréhensions, des incompatibilités de code et une grande difficulté à intégrer les différentes parties du projet.

Le deuxième problème majeur est celui de la distribution des tâches. Nous n'avions pas, en début de projet, les connaissances suffisantes pour évaluer la complexité réelle de chaque composante.

Nous n'avons pas su comment répartir correctement le travail : certaines tâches ont été sous-estimées, d'autres ont été regroupées à tort — comme l'IA et le réseau, qui constituent en réalité deux domaines entièrement distincts nécessitant chacun un investissement considérable. Cette mauvaise répartition a conduit à une concentration du travail sur quelques membres du groupe et à une progression inégale selon les parties du projet.

Ces déséquilibres se sont concrètement matérialisés dans notre rendu final. Nous avons

prévu de livrer dix salles jouables : nous en avons finalement réalisé quatre, dont une intégrant directement les mécaniques d'intelligence artificielle.

Si ce chiffre reste en deçà de nos ambitions initiales, il témoigne néanmoins d'une progression réelle et d'une montée en compétences significative au fil des soutenances. Le système réseau, qui conditionne l'ensemble de l'expérience multijoueur, a dû être développé tardivement, une fois l'intégralité du reste du code stabilisé.

En effet, la couche réseau repose sur des fondations solides — la gestion des salles, les animations des personnages, les mécaniques de jeu — et ne pouvait être implémentée qu'une fois ces éléments finalisés. Ce séquençage, bien que techniquement justifié, a considérablement réduit le temps disponible pour tester et affiner cette composante pourtant centrale dans un jeu coopératif en LAN.

Concernant l'intelligence artificielle, la simulation développée par Julia a finalement pu être intégrée au jeu principal grâce à la contribution de Melissa, qui a conçu et développé une salle exploitant directement ces mécaniques. Cette intégration tardive, bien que réussie, illustre néanmoins les difficultés de coordination que nous avons rencontrées tout au long du projet.

Face à ces difficultés, nous avons identifié des solutions claires pour tout projet futur de cette envergure.

La première leçon est qu'il faut apprendre avant de produire : consacrer les premières semaines à explorer collectivement les outils, faire des petits prototypes ensemble, et s'assurer que chaque membre comprend les bases techniques du projet avant de se lancer dans le développement.

La deuxième leçon est qu'une bonne distribution des tâches ne peut se faire qu'avec une compréhension préalable de leur complexité réelle : il aurait fallu découper le projet en modules précis dès le départ, définir des interfaces claires entre les différentes parties du code, et établir des jalons intermédiaires hebdomadaires plutôt que de tout concentrer sur les dates de soutenance.

Enfin, la troisième leçon concerne la communication au sein de l'équipe : le travail de chaque membre doit être partagé régulièrement et de manière transparente, car dans un projet aussi interdépendant, le retard ou l'absence d'une contribution peut bloquer l'ensemble de la chaîne de développement.

Conclusion globale:

La réalisation de "What's Next?" aura été, pour chacun des membres de l'équipe Boneca Ambalabu, bien plus qu'un simple exercice de programmation.

Ce projet nous a confrontés, tout au long de l'année, à la réalité concrète du développement d'un jeu vidéo : la complexité technique, les imprévus, les compromis, et surtout la nécessité de travailler ensemble de manière coordonnée et efficace.

Sur le plan technique, nous avons su relever les trois grands défis que ce projet imposait. La gestion des ressources graphiques et l'intégration des cartes Tiled dans Pygame ont constitué le socle sur lequel tout le reste repose.

L'implémentation de l'intelligence artificielle des ennemis, à travers l'algorithme A et la machine à états, nous a initiés à une réflexion algorithmique exigeante que nous n'avions jamais eu à mener jusqu'ici, et qui a finalement abouti à une intégration concrète dans le jeu grâce à la salle développée par Melissa.

Enfin, la mise en place de l'architecture réseau TCP, bien que développée tardivement en raison de sa dépendance aux autres composantes du projet, nous a permis de transformer un prototype local en une véritable expérience multijoueur coopérative, répondant aux exigences du cahier des charges.

Ce projet nous a également appris, parfois douloureusement, ce que signifie gérer un projet de longue durée en équipe.

Le manque de familiarisation précoce avec les outils, la difficulté à répartir correctement les tâches sans en connaître la complexité réelle, et les problèmes de communication au sein du groupe ont eu des conséquences concrètes sur notre rendu : quatre salles réalisées au lieu de dix prévues, des sprites de personnages qui ont nécessité de nombreuses tentatives avant d'aboutir à une version exploitable, et un système réseau finalisé dans les dernières semaines du projet.

Ces écarts entre nos ambitions initiales et le résultat final ne sont pas des échecs, mais des indicateurs précieux de ce que nous aurions dû anticiper et mieux organiser dès le départ.

Les leçons que nous en tirons sont claires. Il faut apprendre avant de produire : se familiariser collectivement avec les outils dès le début, faire des prototypes ensemble, et s'assurer que chaque membre comprend les bases techniques avant de se lancer dans le développement.

Il faut également découper le travail en modules précis avec des jalons intermédiaires réguliers, plutôt que de tout concentrer sur les dates de soutenance.

Enfin, dans un projet aussi interdépendant, la communication et le partage régulier du travail de chacun sont absolument indispensables : une contribution en retard ou non transmise peut bloquer l'ensemble de la chaîne de développement.

"What's Next?" n'est pas un jeu parfait. Mais il est le reflet fidèle de ce que cinq étudiants de première année, sans expérience préalable en développement de jeux vidéo, peuvent accomplir lorsqu'ils s'engagent pleinement dans un projet commun et en tirent des enseignements à chaque étape.

La question que pose notre jeu résonne également pour nous en tant que développeurs : cette expérience nous a donné le goût de la création, et nous espérons que les compétences et les leçons acquises ici constitueront le point de départ d'aventures techniques encore plus ambitieuses.

Annexe:

CDCT:



